



Guide de Style et Principes de Design

Édition Étendue

VBA • VB6

Mathieu Guindon

Avril 2026 (2e édition)

PRÉFACE

Que vous soyez un développeur chevronné ou que vous débutiez tout juste votre parcours dans le domaine, Rubberduck (RD) s'avère un atout précieux lorsque vous travaillez avec du code VBA.

Avec plus de 100 inspections, RD excelle à donner des conseils et à proposer des meilleures pratiques pour produire du code plus propre et plus efficace. Si plusieurs de ces inspections donnent des pistes pour écrire un meilleur code, d'autres vont plus loin en aidant à repérer des erreurs avant même que vous ne vous retrouviez en mode débogage.

Et les inspections ne sont que le début de ce que RD a à offrir. Il propose une foule d'autres fonctionnalités pratiques, comme la mise en forme du code, l'indentation et les tests unitaires. Dès que vous l'intégrez à votre processus de développement, il devient difficile d'imaginer travailler sans lui.

Associé à ce guide de style Rubberduck complet, RD ne se contente pas de formuler d'excellentes suggestions : il en explique aussi les raisons. En assimilant ces lignes directrices et en les appliquant, vous vous tracez la voie vers une pratique du développement plus raffinée.

— Wayne Phillips, auteur de *vbWatchDog* et *twinBASIC*

À PROPOS DE L'AUTEUR

Mathieu Guindon a commencé à s'intéresser à la programmation vers l'âge de 12 ans, apprenant avec avidité tout ce qui concernait BASIC 2.0 sur Commodore 64. Tout au long de son adolescence, il s'est essayé à QBASIC, puis plus tard à Visual Basic 4, puis 5, puis VB6, le tout comme passe-temps. Il a ensuite laissé cela de côté pour travailler dans l'industrie du commerce de détail, où il a découvert qu'il pouvait automatiser une grande partie du travail administratif de bureau (le sien et celui de ses collègues) grâce à VBA. Rapidement, il a appris à utiliser le modèle objet d'Excel pour accomplir tout ce qui devait être fait. Quelques années plus tard, jusqu'en 2013, Mathieu s'est impliqué dans la communauté Code Review sur Stack Exchange, et a finalement été élu modérateur. À ce moment-là, il avait déjà rédigé des centaines et des centaines de réponses sur Stack Overflow, et commençait à produire beaucoup de contenu VBA sur Code Review. Au moment où Chris s'est joint à ses efforts pour faire de VBA un tag digne d'un badge sur Code Review — et pendant plusieurs années après — Mathieu était omniprésent sur le tag VBA de Stack Overflow, et a finalement grimpé jusqu'au top 3 des contributeurs de tous les temps de ce tag. À mesure que Rubberduck gagnait en popularité et en crédibilité, l'un de ses principaux contributeurs (également MVP, à l'insu de Mathieu à l'époque!) a nommé Mathieu MVP Microsoft. Le prestigieux prix lui a été remis en janvier 2018 en reconnaissance de ses efforts, et a été renouvelé chaque année jusqu'à ce que la création de contenu l'épuise en 2020 et qu'il ne demande tout simplement pas de renouvellement au-delà de 2021. Aujourd'hui, Mathieu est développeur .NET et SQL, consultant, et programme toujours comme passe-temps. Il est aussi un musicien talentueux (plus qu'il ne l'admettra, en tout cas) qui s'intéresse actuellement à l'harmonica, mais qui a déjà joué de la guitare et a commencé à apprendre un peu de piano « pour le plaisir » il n'y a pas si longtemps.



REMERCIEMENTS SPÉCIAUX

Un merci tout spécial aux contributeurs de Rubberduck : Max, Ben, Vogel, tous les Andrews (+ThunderFrame), Brian, Iven, ainsi que tous les autres — Wayne, Carlos, Rob, Stephen — et tous ceux qui ont contribué à Rubberduck d'une manière ou d'une autre, y compris en signalant des problèmes ou en suggérant des fonctionnalités. Merci à chaque personne qui a « étoilé » le projet sur GitHub, et à tous ceux qui ont écrit des billets ou des critiques au sujet de Rubberduck ou de certaines de ses fonctionnalités. Un immense merci à la communauté et à la plateforme Ko-fi, et tout particulièrement à mes supporters Ko-fi et aux fans de Rubberduck qui se procurent des articles dans la boutique Ko-fi et aident à répandre le canard partout où VBA existe! Merci aussi aux communautés Stack Overflow et Code Review (Stack Exchange) pour leur soutien de la première heure et leur flux constant d'idées de contenu et d'inspections VBA.

Et merci, Chris, d'avoir déclenché tout ça — et d'avoir trouvé le meilleur nom possible pour un projet d'add-in d'IDE.

Ce document n'aurait pas pu exister sans le soutien de ma femme, ni sans la curiosité de mes enfants. Merci pour tout ce temps : je sais que je ne pourrai jamais le « rendre ».

Ce que vous êtes sur le point de lire est essentiellement la synthèse de tout ce que j'ai pu imaginer pour aider les développeurs VBA à acquérir de nouvelles astuces — et des compétences transférables — afin de devenir de meilleurs programmeurs, peu importe le langage utilisé. Ce matériel applique des concepts et des principes modernes à un langage qui est resté figé dans le temps depuis environ 25 ans. Il peut donc bousculer, voire contredire, certains conseils que vous avez reçus par expérience ou lus quelque part sur le web. Je l'admets : tout ce que j'ai construit et écrit se trouve aussi « quelque part sur le web ». Mais ces lignes directrices sont surtout des modernisations, largement inspirées par ce qui fait du bon code VB.NET, conforme aux standards actuels. L'ironie, c'est qu'aucun de ces contenus ne dépend d'une fonctionnalité du langage qui n'existait pas déjà il y a 25 ans.

Programmer en VBA est rempli de pièges — parfois sournois, parfois explosifs. Heureusement, les inspections Rubberduck peuvent en éviter un bon nombre. Et si vous débutez et que vous lisez ce document, vous êtes entre de bonnes mains : suivre ces pratiques et utiliser Rubberduck vous place une longueur (ou deux) d'avance sur quiconque avance sans les outils que vous venez de vous donner. Une grande partie des connaissances nécessaires à certaines inspections a demandé des années à accumuler; elles reflètent aussi l'expérience combinée de plusieurs développeurs VBA chevronnés (actuels et anciens), y compris des MVP Microsoft, passés et présents.

Si vous êtes un développeur VBA expérimenté et que vous découvrez Rubberduck, attendez-vous à être un peu bousculé.

Les améliorations d'IDE apportées par Rubberduck ouvrent un tout nouveau niveau de travail avec un projet VBA, et peuvent rapprocher votre flux de travail de celui d'un développeur logiciel — parce que, que nous en ayons conscience ou non, que nous le voulions ou non, écrire du code VBA fait de nous des programmeurs. Soyons fiers d'apprendre et d'utiliser VBA!

Merci, cher lecteur, de soutenir ce que nous essayons de faire : faire entrer VBA et le VBIDE dans ce siècle!

TABLE DES MATIÈRES

PRÉFACE	2
À PROPOS DE L'AUTEUR	2
REMERCIEMENTS SPÉCIAUX	3
TABLE DES MATIÈRES	4
TERMINOLOGIE ET CONVENTIONS (NORMALISATION)	4
À PROPOS DE RUBBERDUCK	5
HISTORIQUE	5
PHILOSOPHIE	7
ANALYSE STATIQUE DU CODE	8
OUTILS DE NAVIGATION	11
LIGNES DIRECTRICES	15
NOMMAGE	15
PARAMÈTRES ET ARGUMENTS	17
COMMENTAIRES	20
VARIABLES	22
LIAISON TARDIVE (LATE BINDING)	24
EXPLICITÉ	25
GESTION DES ERREURS	26
PROGRAMMATION STRUCTURÉE (PROCÉDURALE)	29
PROGRAMMATION ORIENTÉE OBJET	31
INTERFACES UTILISATEUR	35
CONCEPTION UI	38
SUPPLÉMENTS	41
MOTIFS ARCHITECTURAUX	41
TESTS UNITAIRES	57
PATRONS DE CONCEPTION	63
ANNEXES	69
INDEX	69
GLOSSAIRE	70

TERMINOLOGIE ET CONVENTIONS (NORMALISATION)

Dans ce document, les mots-clés VBA/VB6 et les éléments de syntaxe sont conservés en anglais, tels qu'ils apparaissent dans l'éditeur : Option Explicit, Public/Private, ByRef/ByVal, Sub/Function/Property Get/Let/Set, Set, Enum, On Error, Resume, Debug.Assert, Object, Variant. Les concepts, eux, sont traduits : *early binding* = *liaison anticipée* et *late binding* = *liaison tardive*. Les noms propres (Rubberduck, VBIDE, MSForms, etc.) et les identifiants de code ne sont pas traduits.



À PROPOS DE RUBBERDUCK

En 2009, j'étais déjà un vétéran de VBA quand j'ai commencé à apprendre C# et .NET. Pour moi, le VBIDE était tout ce dont j'avais besoin d'un IDE, et j'aimais qu'il me rappelle de bons souvenirs d'une décennie plus tôt, quand je m'amusais avec VB6 et Visual Studio 6.0. Puis on m'a donné une licence de Microsoft Visual Studio 2010 Ultimate avec ReSharper (R#) de JetBrains, et ça a tout changé.

HISTORIQUE

Naviguer dans Visual Studio 2010 était beaucoup plus simple que dans le bon vieux VBIDE. *IntelliSense*, à lui seul, représentait une amélioration majeure : non seulement on avait l'information sur les paramètres, mais aussi de la documentation xmldoc, générée directement à partir de commentaires dans le code. Les outils de navigation, eux aussi, jouaient dans une autre ligue : ma fonctionnalité préférée de R# est vite devenue la commande *Go to Anything*, et les capacités de refactorisation ont complètement changé ma façon d'aborder des tâches comme *nommer les choses* et nettoyer du code.

Pendant longtemps, utiliser Visual Studio sans ReSharper donnait l'impression qu'il manquait quelque chose : l'add-in faisait des choses que l'IDE *aurait dû* faire, mais qu'il ne fera que bien plus tard, avec l'arrivée de Roslyn — un nouveau compilateur pour .NET/C#, entièrement écrit en C#. C'est à ce moment-là que Visual Studio a commencé à intégrer toutes les fonctionnalités qui faisaient de ReSharper un outil remarquable.

J'ai ensuite relevé d'autres défis, prêt à laisser VBA derrière moi... mais la vie en a décidé autrement.

Vers le milieu de 2014, j'apprenais de nouvelles choses que VBA pouvait faire et que j'ignorais complètement. Rapidement, nous étions une joyeuse bande de passionnés à produire du contenu VBA sur le site *Code Review Stack Exchange*^[1], repoussant l'horizon de notre expérience collective à chaque nouvelle publication. Bientôt, nous appliquions des concepts .NET^[2] en VBA; et en explorant la bibliothèque VBIDE Extensibility, nous avons commencé à jouer avec l'idée d'écrire du code VBA capable de lire, d'écrire et d'invoquer d'autre code VBA. L'étincelle qui a déclenché le projet qui deviendra Rubberduck, c'était un petit add-in Excel : il repérait des méthodes précises à exécuter, les invoquait, puis évaluait le résultat d'un test à partir d'appels à Assert faits dans la procédure invoquée. Porter cette bibliothèque de tests unitaires VBA pour Excel vers C# et .NET nous permettait de nous libérer des contraintes de VBA et d'étendre l'IDE plutôt que l'application hôte. Le faire fonctionner dans tous les hôtes Office a été notre premier défi : nous avons vite découvert que Application.Run n'était pas une évidence.

À l'époque, je ne savais encore rien du *lexing* et du *parsing*. Mais je comprenais une chose : avoir un accès programmatique à tout le code chargé dans l'éditeur signifiait qu'on pouvait, en théorie, écrire du code capable de comprendre la syntaxe VBA — puis informer l'utilisateur des pièges qui font trébucher les débutants. Par exemple, si Option Explicit manque dans un module, c'est très facile à détecter, et à signaler dans une fenêtre d'outils personnalisée de l'IDE.

Mais ce n'était pas suffisant. Je voulais que Rubberduck soit le *ReSharper pour VBA* : l'add-in de référence que tout développeur VBA sérieux garde dans sa boîte à outils, l'incontournable gratuit à l'« aube » de VBA — si l'on qualifie le



début des années 2000 de son « âge d'or ». Je voulais que Rubberduck *comprenne* VBA assez profondément pour nous permettre d'implémenter le saint Graal des fonctionnalités de refactorisation : *extract method*. Avec R#, on pouvait sélectionner n'importe quel nombre de lignes, n'importe où dans une portée, puis, avec un raccourci clavier, déplacer ce bloc dans une nouvelle procédure bien nommée et remplacer la sélection par un appel à la méthode extraite. L'outil déplaçait automatiquement les variables qui pouvaient l'être et devinait quels paramètres et valeurs de sortie étaient nécessaires. Tout cela — et même simplement renommer correctement n'importe quoi — exige un *métamodèle* exact de ce qui se passe. La moindre erreur dans ce modèle peut produire un refactor qui casse le code, ce qui annule entièrement l'intérêt : une opération de *refactorisation* ne change pas *ce que* fait le code, seulement la manière d'y parvenir, le *comment*.

Ce qui est amusant, c'est qu'il existe aujourd'hui toute une génération de développeurs pour qui ce genre de magie fait simplement partie des fonctionnalités normales de *n'importe quel* IDE — et ils ont raison. En 2024, un bon IDE offre une excellente expérience développeur : il facilite la navigation, guide les débutants sans les submerger, et rend ses fonctionnalités faciles à découvrir et intuitives à utiliser.

Sauf si vous êtes « la personne des macros » au bureau, coincée dans le VBE. Là, vous êtes resté en 1999... à moins que — à moins que vous ne connaissiez Rubberduck.

Rubberduck n'est pas sans défauts : à partir de la version 2.5.2, l'add-in consomme *beaucoup* de ressources et peut mal se comporter avec des projets très volumineux. Son démarrage ajoute un délai notable au chargement du VBE, et l'intégration COM vient avec son lot de subtilités et de problèmes potentiels que nous n'avions pas anticipés, comme le mécanisme d'ancrage des fenêtres d'outils du VBIDE. Parfois cela entraîne un léger ralentissement; parfois Rubberduck ne se ferme pas correctement et déclenche une exception au moment où l'application hôte se ferme. Malgré tout, l'expérience du VBIDE avec Rubberduck est nettement meilleure que sans.

Le travail sur la prochaine évolution de Rubberduck a commencé : elle corrigera la plupart (sinon toutes) des limitations les plus sérieuses de la branche 2.x en implémentant un *language server* pour VBA, ce qui déplacera hors du processus hôte une grande partie de ce qui consomme des ressources. Le jour où elle sortira, cette prochaine évolution sera tout ce que nous avons toujours voulu que Rubberduck devienne. Et même si plusieurs choses changeront radicalement, la façon dont les développeurs VBA devraient utiliser Rubberduck et écrire du code VBA, elle, ne changera pas. Les lignes directrices et bonnes pratiques décrites dans ce document restent donc pertinentes, que vous utilisiez Rubberduck 2.x, que vous lisiez ceci dans quelques années avec Rubberduck 4.x, ou que vous n'utilisiez pas du tout Rubberduck.



PHILOSOPHIE

L'une des toutes premières inspections implémentées dans Rubberduck fut l'inspection Option Explicit. D'accord, c'était en partie parce qu'elle était simple à mettre en œuvre, même avant que nous ayons un *vrai* analyseur syntaxique... mais l'idée de fond était (et reste) que *personne ne sait tout*. C'est en rassemblant nos connaissances qu'on devient plus solides, et c'est pourquoi l'analyse statique de Rubberduck explique le raisonnement derrière chaque résultat d'inspection : Rubberduck signale beaucoup de choses dont je n'avais aucune idée il y a 10 ou 15 ans. Cela ne m'a jamais empêché (et ne vous empêchera pas non plus) d'écrire du code VBA qui « fonctionnait » très bien... sauf quand ce n'était pas le cas. Mais, qu'on l'accepte ou non, une macro écrite en VBA est un ensemble d'instructions exécutables : c'est donc un *programme*; écrire une macro, c'est donc faire de la *programmation*; et cela fait de nous des *programmeurs*. Et nous nous le devons : viser du code de qualité — pas seulement du code qui *marche*, mais du code qui se *lit* facilement, et qui est agréable à parcourir, maintenir et faire évoluer.

Le fait d'être des programmeurs qui écrivent et maintiennent du code VBA nous distingue, notamment parce que le langage ne disparaîtra pas, tandis que l'IDE devient, au fil des années, de plus en plus désuet et pauvre en fonctionnalités. Pourtant, si le volume de questions sur VBA sur Stack Overflow veut dire quelque chose, VBA est encore bien vivant, encore largement appris — et c'est précisément là que Rubberduck et l'analyse statique ont leur place.

Quand j'ai commencé à apprendre .NET et C# il y a plus de dix ans, il y avait une nouveauté enthousiasmante : une fonctionnalité de langage qu'on appelait LINQ pour *Language-Integrated-Query*, qui permettait d'interroger des collections d'objets presque comme on interroge une base de données. C'était excellent (et ça l'est toujours). Pour rendre cela possible, le compilateur C#, le framework .NET et le runtime lui-même ont dû évoluer de façon intéressante — [Jon Skeet l'explique en détail](#). Le point, c'est que cette nouvelle syntaxe pouvait dérouter au début, et qu'elle amenait des implications nouvelles et importantes (closures, exécution différée). L'entreprise pour laquelle je travaillais nous a donné une licence [ReSharper](#) : c'est ainsi — et à ce moment-là — que j'ai découvert à quel point des outils d'analyse statique complets et précis pouvaient être un formidable outil d'apprentissage.

Et j'aimerais que Rubberduck soit comme ça : un compagnon qui regarde votre code et vous révèle des détails, des indices du genre « saviez-vous que cette affectation conditionnelle pourrait être simplifiée? », ou encore « si cette condition était inversée, ce bloc vide deviendrait inutile ».

Peut-être que nous ne sommes pas d'accord au sujet de la *notation hongroise*, et c'est correct : Rubberduck veut que vous puissiez la repérer et la renommer si c'est ce que vous souhaitez, et vous pouvez aussi désactiver cette inspection à tout moment. Mais je pense que l'outil *doit* vous dire ce qu'est la notation *Systems Hungarian* lorsqu'il la signale — et peut-être aussi expliquer ce qu'est *Apps Hungarian* en donnant des exemples, parce que *Apps Hungarian* est vraiment utile et porteuse de sens (pensez au préfixe o-pour-OneBased, ou aux préfixes src-pour-Source et dst-pour-Destination). En revanche, str-pour-String, lng-pour-Long, o-pour-Object (c'est-à-dire les types intrinsèques) sont d'une autre nature : cela ne dit rien du *rôle* ou du *but* de l'identifiant.

Rubberduck signale des constructions et mots-clés obsolètes — des déclarations Global, l'instruction On Local Error, l'usage explicite du mot-clé Call, ou encore les boucles While...Wend. Rien de tout cela n'a de raison d'exister dans du VBA neuf, écrit aujourd'hui; et les correctifs rapides peuvent les transformer en déclarations Public, en instructions On Error, en appels implicites (sans le mot-clé Call), ou encore en boucles Do While...Loop.

Rubberduck vise à moderniser le VBE et à orienter votre façon de programmer vers un code objectivement — et *mesurablement* — meilleur. En 2024, le projet a marqué une décennie complète depuis ses débuts, et cela a été

souligné avec quelques articles (« swag ») sur une boutique Ko-fi qui est depuis fermée, mais qui fait néanmoins partie de l'histoire du projet. Cette nouvelle édition du guide est distribuée directement sur rubberduckvba.ca.



Figure 1 — Article Ko-fi historique : autocollant découpé « Célébration 10 ans Rubberduck » (boutique Ko-fi aujourd'hui fermée).



ANALYSE STATIQUE DU CODE

Parmi les premières idées à prendre forme, il y avait celle de laisser Rubberduck analyser le code qu'il interprète, et mettre en évidence des problèmes potentiels. Beaucoup d'inspections encouragent des pratiques de codage objectivement souhaitables; d'autres relèvent davantage d'observations plus subjectives et discutables. Mais le simple fait de pouvoir balayer une base de code et signaler ces éléments aide à maintenir un style *cohérent*. Et quand la subjectivité entre en jeu, ce qui compte vraiment, c'est que le style reste *cohérent*.

Si vous activez Rubberduck sur un projet VBA hérité contenant une quantité significative de code, il y a de fortes chances que l'analyse statique génère *une tonne* de résultats d'inspection pour toutes sortes de petites choses banales. Votre objectif devrait-il être de *tout corriger par correctifs rapides* et d'obtenir un code qui ne déclenche *aucun* résultat d'inspection Rubberduck?

De façon peut-être surprenante, la réponse est un « non » catégorique, tout simplement parce qu'un outil automatisé se trompera... parfois. La présence d'un diagnostic / d'un résultat d'inspection indique un *problème potentiel* dans le code *tel que Rubberduck le comprend*. S'il y a une faille dans cette compréhension, certaines inspections peuvent produire des faux positifs, et/ou des correctifs rapides et des refactorisations peuvent être indisponibles, ou ne pas fonctionner comme prévu.

NIVEAUX DE SÉVÉRITÉ

Dans Rubberduck, chaque inspection possède un *niveau de sévérité* configurable, qui est par défaut à **Warning** pour la plupart des inspections (c'est la valeur par défaut — sauf indication contraire — pour toutes les inspections Rubberduck) :

- **Error** indique un problème potentiel auquel vous voudrez probablement prêter attention immédiatement, parce qu'il pourrait être (ou provoquer) un bogue. Si les résultats d'inspection étaient rendus directement dans l'éditeur (ce sera le cas dans RD3), ce seraient des soulignements ondulés rouges qui crient « regarde-moi ».

- **Warning** indique un problème potentiel dont vous devriez être conscient.
- **Suggestion** est généralement utilisé pour signaler des opportunités d'amélioration, pas nécessairement des problèmes.
- **Hint** sert aussi à signaler diverses opportunités non problématiques. Si les résultats d'inspection étaient rendus directement dans l'éditeur, ce serait un soulignement discret en pointillés, avec un texte au survol.
- **DoNotShow** désactive l'inspection : non seulement ses résultats ne seront pas affichés, mais ils ne seront même pas *générés*.

Par défaut, Rubberduck est configuré pour exécuter toutes les inspections (actuellement plus de 100), à quelques exceptions soigneusement choisies — des inspections qui signaleraient exactement l'inverse de ce qu'une autre inspection activée signale déjà. Par exemple, nous livrons l'inspection *implicit ByRef modifier* activée (au niveau **Hint**), mais l'inspection *redundant ByRef modifier* est désactivée tant que vous ne lui assignez pas un niveau de sévérité autre que **DoNotShow**. Cela évite de « corriger » un résultat d'inspection pour en obtenir immédiatement un nouveau qui signale exactement l'inverse — ce qui serait, à juste titre, déroutant pour des utilisateurs qui ne sont pas familiers avec les outils d'analyse statique.

Les inspections sont-elles, d'une certaine façon, « imprégnées » de la connaissance du niveau auquel vous devriez les traiter — erreur, avertissement, ou simples indications et suggestions? Parfois, oui. *Missing Option Explicit* devrait faire l'objet d'un consensus clair au niveau **Error**. À l'inverse, savoir si un *implicit default member call* ou l'utilisation d'un *empty string literal* devrait être un **Warning**, un **Hint**, ou même être rapporté tout court, dépend probablement davantage de votre aisance / expérience avec VBA/VB6, ou même d'une préférence personnelle; ce qui importe, c'est que l'outil d'analyse statique vous apprend quelque chose sur le code, que le code seul ne dit pas nécessairement.

Les niveaux de sévérité devraient être revus avec soin afin de refléter votre style et vos préférences.

MÉTRIQUES DE CODE

Rubberduck pourrait compter le nombre de lignes d'une procédure et produire un résultat d'inspection au-delà d'un seuil configurable. En fait, plusieurs éléments se mettent lentement en place pour que cela finisse par arriver. Mais on ne voudrait surtout pas que vous découpiez arbitrairement une procédure à 20 lignes « parce qu'une inspection l'a dit »! Rubberduck peut mesurer le nombre de lignes, les niveaux d'imbrication, et la *cyclomatic complexity* (complexité cyclomatique). Ces métriques peuvent servir à repérer des zones problématiques dans une base de code et à découper méthodiquement de gros problèmes complexes en problèmes mesurablement plus petits et plus simples.

Nombre de lignes compte simplement le nombre de lignes. À terme, cela pourrait s'étendre au nombre d'*instructions* et de *commentaires*, peut-être avec des pourcentages; par exemple, 10 % de commentaires est probablement un bon signe. Mais aucun outil ne vous dira que 'increments i est un commentaire inutile, et même les meilleurs outils auront de la difficulté à distinguer un énorme bandeau du type 'the following chunk of code does XYZ d'un commentaire réellement *utile*. La sagesse populaire est de garder cette *métrique de lignes* aussi basse que possible, mais il ne faut pas le faire au détriment de la lisibilité.

Niveaux d'imbrication comptent le nombre de... niveaux d'imbrication. Imbriquer deux boucles For...Next pour parcourir un tableau 2D (ou une Range de cellules) de haut en bas et de gauche à droite est probablement raisonnable; au-delà, il est souvent préférable de rendre l'imbrication implicite au moyen d'un appel de procédure. Règle générale : extraire le corps d'une boucle dans sa propre procédure paramétrée est presque toujours une bonne idée. Le code en forme de flèche (*arrow-shaped code*) s'aplatit, le nombre de lignes baisse, et les procédures deviennent plus spécialisées et ont moins de raisons d'échouer.

Complexité cyclomatique calcule essentiellement le nombre de chemins d'exécution indépendants dans chaque procédure^[1]. Une procédure dont la complexité cyclomatique dépasse 5 est plus difficile à suivre qu'une procédure dont la complexité est de 1 ou 2; mais il n'est pas rare de voir une « procédure dieu » (God procedure) avec des boucles et des conditions imbriquées mesurer dans les 40 et plus.

La fonctionnalité de *métriques de code* finira par recevoir toute l'attention qu'elle mérite (elle sera probablement rétroportée depuis le dépôt RD3), mais comme pour les inspections, l'idée générale est de mettre en évidence les procédures qui pourraient être plus difficiles à maintenir que nécessaire, et d'inciter nos utilisateurs à :

- Écrire davantage de procédures plus petites et plus spécialisées.
- Passer des paramètres entre les procédures au lieu d'utiliser des variables globales.
- Avoir davantage de modules plus petits et plus *cohésifs*.



OUTILS DE NAVIGATION

Vous l'avez peut-être remarqué (ou pas), mais l'éditeur VBA (VBE) vous pousse dans la direction *exactement opposée*, parce que...

- Avoir moins de procédures, plus grosses et plus généralistes vous place dans un état d'esprit de *scripting*.
- Utiliser des variables globales au lieu de faire circuler l'état par paramètres est souvent perçu comme plus simple.
- Avoir moins de modules, plus gros et plus généralistes rend le partage de code entre projets plus simple, et c'est parfois plus facile pour retrouver quelque chose dans le *Project Explorer*.

Si vous écrivez un petit script, vous pouvez — et vous devriez probablement — fonctionner comme ça.

Mais si, comme moi, vous poussez VBA à faire des choses pour lesquelles il n'a pas vraiment été conçu, vous finissez par maintenir de vraies *applications* qui pourraient tout aussi bien être écrites dans n'importe quel autre langage... et vous le faites en VBA parce que *vos raisons sont valides, quelles qu'elles soient*.

Et c'est là que ça devient un problème, parce que le VBE semble activement *ne pas* vouloir que vous écriviez du code orienté objet correctement : ses outils de navigation rendent très difficile le travail dans un projet composé de *nombreux* petits modules — sans parler d'un projet POO qui utilise des interfaces explicites et des niveaux d'abstraction élevés.

Rubberduck lève à peu près toutes les limitations de l'IDE qui empêchent de traiter un projet VBA comme autre chose qu'un simple *script d'automatisation*. Vous pouvez maintenant avoir un projet avec 135 modules de classe, soigneusement organisés par fonctionnalité dans des dossiers qui peuvent contenir n'importe quel type de module; ainsi, un UserForm peut apparaître juste à côté des classes qui l'utilisent, sans recourir à des schémas de préfixes disgracieux. Vous pouvez faire un clic droit sur une interface abstraite (ou sur l'un de ses membres) et trouver rapidement toutes les classes qui l'implémentent. Vous disposez d'une commande *Find symbol* qui permet de naviguer rapidement vers littéralement n'importe quoi qui a un nom, n'importe où dans le projet. Vous êtes curieux des détails d'implémentation d'une procédure, mais vous ne voulez pas casser votre rythme en y naviguant? *Peek definition* vous y amène sans quitter l'endroit où vous étiez.



Figure 2 — Peek definition affiche un panneau flottant montrant, de façon pratique, le code source du module ou du membre défini par l'utilisateur que vous avez sélectionné.

Rubberduck peut lister toutes les références à une déclaration donnée (qu'elle soit définie par l'utilisateur ou non) et les afficher avec leur emplacement et leur contexte. Vous pouvez faire un clic droit sur n'importe quel identifiant et sélectionner « Find all references » dans le menu Rubberduck; ou encore placer le curseur sur l'identifiant et cliquer sur le bouton « n references » dans la barre d'outils Rubberduck pour obtenir le même résultat :

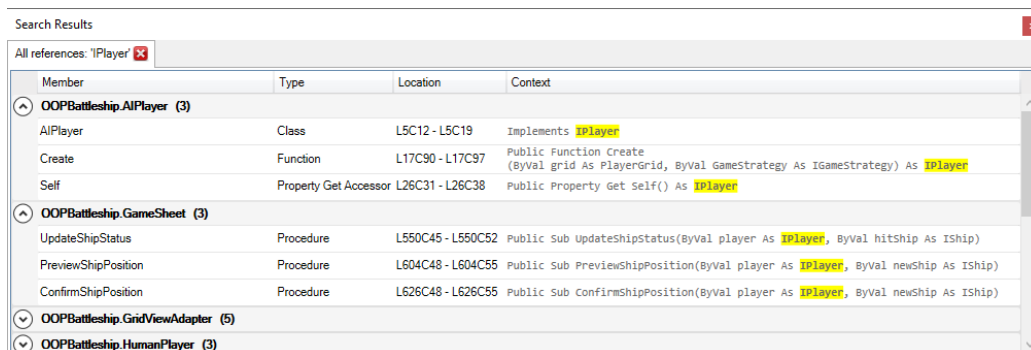


Figure 3 — La fenêtre d'outils Search Results affiche les résultats de « Find all references ».

De la même façon, le découplage obtenu dans des scénarios avancés impliquant des interfaces abstraites signifie que naviguer vers la définition d'une méthode à partir d'un de ses sites d'appel (F2 dans le VBE) vous amène sur le membre de l'interface abstraite; il faut alors rechercher l'instruction Implements qui mentionne le nom de l'interface pour faire défiler les implémentations — ouch! Avec Rubberduck, il suffit de choisir « Find all implementations » pour arriver rapidement au code que vous voulez consulter.

```
'@Description("Attempts to make a hit on the enemy grid.")
Public Function Play(ByVal enemyGrid As PlayerGrid) As IGridCoord
End Function
```

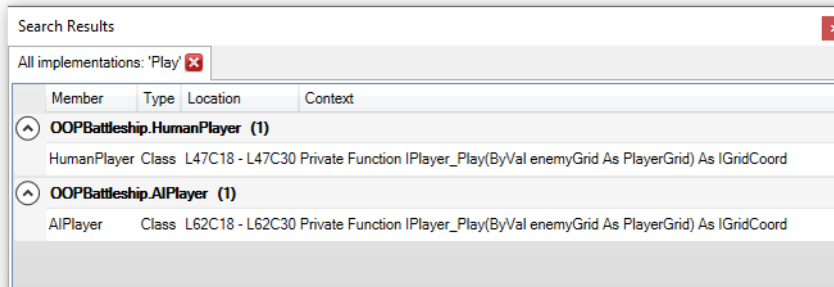


Figure 4 — La commande Find all Implementations est extrêmement utile dans les projets orientés objet qui tirent parti du polymorphisme via des interfaces abstraites : repérez et naviguez rapidement vers n'importe quelle implémentation de n'importe quelle interface (classe ou membre).

Le *Project Explorer* du VBE vise à offrir une vue d'ensemble du projet en regroupant les modules par *type de module*. C'est très bien pour un petit script qui peut s'en tirer avec un petit nombre de composants, mais cela rend la gestion des projets plus volumineux beaucoup plus difficile.

Avec le *Code Explorer* de Rubberduck, vous pouvez descendre jusqu'au niveau des membres (et afficher/masquer leurs signatures), et regrouper les modules par *fonctionnalité* grâce à une hiérarchie de dossiers entièrement personnalisable (les dossiers sont des métadonnées de module que vous contrôlez via des commentaires d'annotation).

Rien que cela suffit à justifier la fonctionnalité, mais avec le temps, les *références de bibliothèques* et les *références de projet* ont obtenu leurs propres nœuds dans l'arborescence, ce qui permet de voir d'un coup d'œil tout ce que le projet référence... et l'expérience d'ajout/suppression de références est, elle aussi, grandement améliorée.



L'arborescence peut être filtrée avec une zone de recherche. Vous pouvez exécuter des commandes au niveau du projet, du module et du membre sans quitter l'endroit où vous êtes, et la sélection d'un nœud affiche ses métadonnées (type de nœud, nom d'identifiant, docstring/description, etc.).

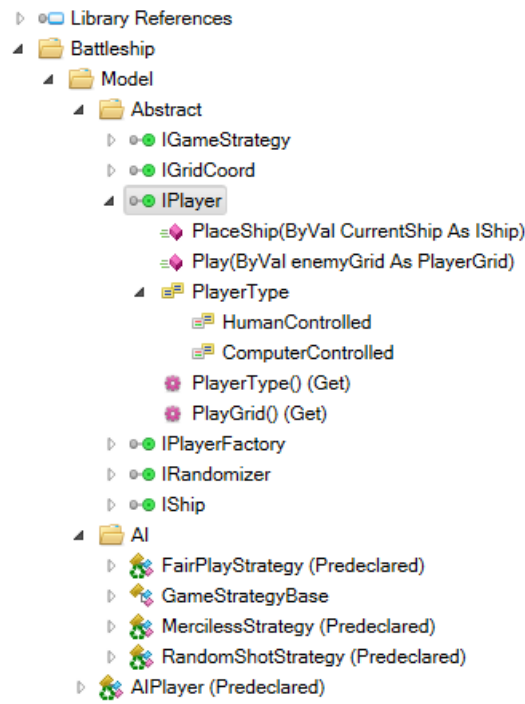


Figure 5 — Le Code Explorer distance nettement le Project Explorer du VBE, sans appel.

Ces améliorations de navigation simplifient énormément la circulation dans un projet de toute taille, même si certaines peuvent sembler un peu *surfaites* dans un petit projet, et que d'autres ne deviennent vraiment utiles que dans des scénarios POO plus avancés. Cela dit, disposer de plus qu'une simple recherche textuelle pour retrouver des éléments est très utile, et les fonctionnalités avancées garantissent que l'IDE ne deviendra pas un obstacle à votre apprentissage.

Rubberduck est plus qu'un outil : c'est une ode aux possibilités sous-explorées de VBA, une prise de position qui affirme que VBA est tout autant un langage de programmation « réel » que les autres; et l'utiliser à son plein potentiel implique, en effet, une nouvelle itération de ce que devraient être les bonnes pratiques en VBA.





GUIDELINES

LIGNES DIRECTRICES

S'il n'y avait qu'un seul principe général pour guider tout le reste, ce serait *d'écrire du code qui fait ce qu'il annonce, et qui annonce clairement ce qu'il fait*. Tout le reste en découle.

De nombreuses caractéristiques du langage autrefois considérées comme pratiques sont aujourd'hui obsolètes : l'instruction `Def[Type]` en est un bon exemple. À l'époque où elle a été introduite, il était probablement utile de pouvoir typer implicitement des variables à partir du premier caractère de leur identifiant. Les schémas de préfixes de notation hongroise viennent en partie de cette époque, tout comme la numérotation des lignes et l'idée que toutes les variables d'une portée devraient être déclarées tout en haut de celle-ci.

Le principe directeur des pratiques de codage de l'époque semblait surtout orienté vers la facilité d'écriture du code : types implicites, modificateurs implicites, conversions implicites, appels de membres implicites, noms d'identifiants courts (ils sont encore plus courts sans les voyelles!)... Aujourd'hui, les pratiques modernes visent plutôt à rendre le code plus facile à *lire*. Parce que programmer, oui, c'est écrire du code, mais c'est surtout *lire* et *comprendre* du code — et éliminer les ambiguïtés.

Vous pourriez ne pas être d'accord avec certaines des recommandations présentées ici, et ce n'est pas un problème : ce sont des *lignes directrices*, pas des règles absolues. Je présente néanmoins une justification pour chacune d'elles, en assumant leur part de subjectivité — et le fait que ces choix reflètent sans gêne mon expérience et mes préférences.

NOMMAGE

Bien nommer les choses est plus difficile qu'il n'y paraît, mais le temps investi pour trouver, chaque fois, le meilleur identifiant possible est largement rentabilisé. Cette capacité évolue avec l'expérience : au début, on est moins porté vers l'abstraction et on a tendance à nommer les choses d'après ce qu'on voit — « j'ai besoin d'une Range », alors on l'appelle `rng1` et cela peut sembler suffisant. Puis, à mesure qu'on progresse, on se rend compte que `rng1` et `rng2` ne décrivent rien, et on commence à préférer des noms plus expressifs, comme `SourceRange` et `DestinationRange`. Les noms de procédures suivent la même trajectoire, de `Macro1` à `CreateInvoice`. Il fut un temps où je ne renommais pas les contrôles de formulaire qui n'avaient ni événements ni code associé. Le résultat : d'un côté, une liste de membres remplie de `txtSomething`, et de l'autre, des contrôles restés à `Label42` — et la dernière chose qu'on veut dans du code-behind, c'est un identifiant sans signification. Nommez tout ce qui peut être nommé, qu'il s'agisse d'un contrôle, d'une forme sur une feuille de calcul ou de n'importe quel objet avec lequel le code doit interagir : si cela a un rôle, cela mérite un nom.

Historiquement, les contrôles de formulaire reçoivent souvent un préfixe de 2 ou 3 lettres qui indique leur type (un peu comme une notation hongroise, mais appliquée aux contrôles). Cela peut aider à reconnaître rapidement qu'on regarde un contrôle d'entrée qui expose une propriété `Value`... mais, à mon avis, le type de contrôle n'a pas besoin de faire partie de son nom. Par exemple, `txtSomething` peut devenir `SomethingBox`, où `Box` suggère simplement une saisie utilisateur. La partie « `Something` » devrait être assez descriptive : `StartDateBox` et `PricePointBox` parlent d'eux-mêmes. Et les gestionnaires d'événements correspondants auront alors des noms cohérents, par exemple `StartDateBox_Change` ou `PricePointBox_Change`, conformément aux conventions de nommage de VBA.

MAJUSCULES

Pendant longtemps, j'ai essayé d'utiliser `camelCase` pour les variables locales et les paramètres, et `PascalCase` pour les modules et leurs membres. Le problème, c'est que VBA est insensible à la casse : il ne fait pas la différence entre `doSomething` et `DoSomething`. Si deux déclarations ont le « même » identifiant, la dernière enregistrée dans la table interne des noms « gagne » et devient la graphie que l'éditeur réutilise partout. Résultat : il devient difficile de rester cohérent, et on se retrouve à chercher des contournements (préfixes arbitraires, etc.) juste pour éviter de « casser » la casse d'un nom — ce qui finit par être une distraction, parce que le langage, lui, s'en fiche. C'est pour ça que j'ai fini par abandonner l'idée de distinguer les noms uniquement par la casse en VBA. Cela dit, je ne blâme personne d'essayer : l'important, c'est la cohérence, et il y a bien plus important que la casse du premier caractère d'un identifiant.

OBJECTIF

Nommer des choses peut être difficile parfois, mais étant donné qu'on lit n'importe quelle ligne de code beaucoup plus souvent qu'on ne l'écrit, prendre le temps de réfléchir au *but* d'une abstraction (que ce soit une variable, une procédure, une classe ou un projet) lors du nom, est toujours un bon investissement.

Vous avez une feuille *de résumé*, pas une « feuille 1 ». Il y a un *bouton accepter* et une *étiquette d'instructions* sur ce formulaire, pas un « Command2 » et un « Label8 ». Si vous avez besoin d'un commentaire pour expliquer ce qu'il y a derrière un nom identifiant, c'est probablement un nom qui n'est pas assez descriptif.

C'est pourquoi vous voulez des noms d'identification faciles à lire et qui ne vous obligent pas à faire une carte mentale de ce qui est quoi, parce qu'ils sont explicites. Pour la même raison, évitez *de changer la signification* d'un identifiant à tout moment pendant l'exécution : il y a amplement de place pour chaque variable dont vous pourriez avoir besoin; évitez de réutiliser un identifiant non seulement dans une procédure, mais dans toute votre base de code.

Évitez de déclarer une variable pour un but *puis de l'utiliser pour un autre*; une variable aléatoire pour contenir une référence d'objet Range rend la réutilisation très facile (voire tentante), contrairement à un nom plus spécifique comme `InvoiceDateCell`. Un identifiant, un but. Si vous tombez sur un nom d'identifiant donné dans votre base de code, il devrait signifier la même chose partout, pour que vous puissiez savoir ce que vous regardez sans avoir à revenir en arrière jusqu'à l'endroit où il a été attribué pour la dernière fois (petite parenthèse, c'est beaucoup plus facile à faire pour les locaux que pour les globals), et pour comprendre la signification d'une expression pour comprendre ce que signifie le nom : Le but de quoi que ce soit devrait toujours être aussi évident que possible rien que par son nom.

RENOMMAGE

Nommer est déjà assez difficile, renommer les choses devrait être facile. Avec la fonctionnalité de renommage (`Ctrl+Shift+R`) : vous pouvez renommer n'importe quel identifiant en toute sécurité *une fois*, et toutes les références à l'identifiant sont automatiquement mises à jour. Sans outil de refactorisation, le renommage d'un contrôle de formulaire ne peut se faire qu'à partir de la fenêtre d'outils *Propriétés* (`F4`), et faire cela casse instantanément tous les gestionnaires d'événements qui y sont liés; renommer une variable à la main peut être fastidieux, et renommer une variable d'une seule lettre comme `a` ou `i` avec une recherche/remplacement (`Ctrl+H`) peut rapidement devenir bizarre si la procédure contient des commentaires. Rubberduck connaît l'emplacement exact de chaque référence à chaque identifiant dans votre projet, donc si vous avez un module avec deux procédures qui déclarent chacune un `localThing`, lorsque vous renommez la variable locale `localThing` Dans la première procédure, vous n'affecterez pas



le `localThing` dans l'autre procédure. Mais si vous renommez `CommandButton1` à `OkButton`, alors `CommandButton1_Click()` devient automatiquement `OkButton_Click()`.

N'hésitez jamais à renommer un mauvais identifiant que vous croisez; soyez impitoyable. Nommer les choses selon leur but plutôt que leur nature (on se fiche qu'un montant soit une valeur de monnaie 64 bits, mais on veut pouvoir distinguer les ventes des montants de coût). Est-ce que quelque chose a besoin de plus de voyelles, ou un meilleur nom vous vient à l'esprit en lisant le code? Renomme-le; Éliminez la distraction et laissez votre esprit se concentrer sur le contenu plutôt que sur son contenant!

EN RÉSUMÉ

- Utilisez `PascalCase` si vous le souhaitez. Utilisez `camelCase` si vous le souhaitez. Ce qui compte, c'est la cohérence; et, dans un langage insensible à la casse qui ne stocke qu'une seule graphie par identifiant, il est souvent plus simple d'utiliser `PascalCase` partout, puis de passer à des choses plus intéressantes, comme les tabulations plutôt que les espaces.
- Évitez `_` soulignés dans les noms d'identifiants, *surtout* dans les noms des procédures ou des membres.
 - Les caractères soulignés peuvent provoquer des *erreurs de compilation* lorsqu'ils sont utilisés dans les noms de contrôles (accès via la collection `Controls`), et ils nuisent souvent à la lisibilité.
- Utilisez des noms significatifs qui peuvent être prononcés.
- Évitez les schémas de suppression arbitraire des voyelles et les préfixes de type *Systems Hungarian*.
- Une série de variables avec un suffixe numérique est une occasion manquée d'utiliser un tableau.
- Un bon nom d'identifiant est assez descriptif pour ne pas avoir besoin d'un commentaire explicatif.
- Utilisez un nom descriptif qui commence par un verbe pour les procédures `Sub` et `Function`.
 - Note : si vous utilisez des *instructions d'appel explicites*, vous pourriez être tenté d'utiliser un nom descriptif à la place (pas `Macro1!`), puisque « *call* » devient le verbe/l'action; pensez à choisir `PrepareWeeklySalesReport` plutôt que `WeeklySalesReport`.
- Utilisez un nom descriptif (un nom) pour les procédures et modules de propriété.
- Pour les propriétés des objets, pensez à les nommer d'après le type d'objet qu'ils retournent, comme `Excel.Worksheet.Range` retourne un objet `Range`, ou comme `ADODB.Recordset.Fields` retourne un objet `Fields`.
- Nommez correctement tout ce avec quoi le code doit interagir : si une forme rectangulaire arrondie est attachée à une macro `DoSomething`, le nom par défaut « `Rounded Rectangle 1` » devrait être changé en « `DoSomethingButton` » ou quelque chose qui nous indique son but. Cela inclut aussi tous les contrôles sur un concepteur `UserForm`. `CommandButton12` est inutile; `SearchButton` est bien meilleur. Envisagez aussi de nommer les contrôles qui *n'interagissent pas* nécessairement avec le code : le code futur pourrait, et l'auteur de ce code futur appréciera que le panneau du bas s'appelle `BottomPanel` et non `Label34`.
- N'hésitez jamais à renommer n'importe quel identifiant pour lequel vous trouvez un nom meilleur et plus descriptif.



PARAMÈTRES ET ARGUMENTS

Lorsque vous commencez à décomposer les grandes procédures en plus petites, vous découvrez rapidement la nécessité de transmettre l'état et les données entre les procédures. Faire passer une déclaration locale au niveau du

module est une façon de procéder : toutes les procédures d'un module ont accès à toutes les déclarations au niveau du module dans ce module.

Mais ce n'est pas parce que deux procédures (ou plus) d'un même module ont besoin d'une valeur particulière qu'il est judicieux de déclarer cette valeur au niveau du module : les *niveaux d'abstraction* doivent rester cohérents, et les déclarations d'un module devraient être étroitement liées. Par conséquent, il est souvent préférable de garder les choses aussi ciblées que possible, et de faire circuler l'état et les données *par paramètres*.

Un *paramètre* est une déclaration locale qui fait partie de la *signature* d'une procédure (qu'il s'agisse d'une Sub, d'une Function, ou d'une procédure Property). À l'exécution, la valeur d'un paramètre est fournie à la procédure au moyen d'une expression d'*argument*, au point d'appel de la procédure. Les procédures Property Let et Property Set doivent avoir un paramètre supplémentaire représentant la valeur (ou la référence) assignée à la propriété. Ce paramètre est souvent nommé Value, NewValue, ou RHS (pour *right-hand side*, le « côté droit » de l'affectation).

Une source fréquente de confusion concerne l'usage des parenthèses lorsqu'on appelle une procédure. Pour comprendre quand elles sont requises, il faut d'abord comprendre la syntaxe d'un appel de procédure. En pratique, on appelle une procédure pour deux raisons : (1) exécuter une série d'instructions ayant des effets de bord, qui se termine soit avec succès, soit par une erreur; (2) récupérer une valeur de retour (fonction ou propriété), ce qui n'est généralement pas un appel « à effet de bord ».

À moins que nous ne souhaitions une valeur de retour, il ne faut pas fournir de parenthèses :

Nom de la procédure [*ArgumentsList*]

Où *ArgumentsList* est une liste optionnelle, séparée par des virgules, d'expressions d'arguments qui sont évaluées juste avant que leurs valeurs ne soient placées sur la pile, afin que la procédure appelée puisse les récupérer.

Une expression d'argument peut être une valeur littérale, une variable, un appel de fonction, un objet, une propriété d'objet, ou une expression plus complexe combinant plusieurs valeurs et fonctions. Par exemple, lorsque vous écrivez MsgBox "Hello", la valeur littérale de la chaîne "Hello" est transmise au paramètre Prompt de la fonction MsgBox. Si vous écrivez MsgBox msg, la variable msg est évaluée et sa valeur est transmise à la fonction. Et si vous écrivez MsgBox "Hello " & name, la valeur contenue dans name est concaténée avec la valeur littérale de "Hello ", et la chaîne résultante est celle que la fonction reçoit.

On pourrait mettre des parenthèses autour de chaque expression d'argument, mais la liste d'arguments, elle, n'est pas entourée de parenthèses. Le VBE le suggère subtilement en imposant un espace entre le nom de la procédure et l'expression du premier argument :

```
MsgBox ("Bonjour" & Nom), ("Titre")
```

Traiter ces parenthèses comme si elles faisaient partie de la liste d'arguments entraîne une erreur de compilation :

```
MsgBox ("Bonjour" & Nom, "Titre")
```

... parce que l'expression entre parenthèses n'a pas de sens : elle ne peut pas être évaluée, et donc VBA ne peut pas déterminer comment la faire correspondre à des paramètres distincts.



Mais que se passe-t-il si vous *voulez* capturer une valeur de retour? Dans ce cas, les parenthèses autour de la liste d'arguments sont requises. Le VBE le suggère aussi, en collant systématiquement la parenthèse ouvrante au nom de la procédure :

```
PromptResult = MsgBox(Msg, Title, vbYesNo)
```

La fonction `MsgBox` retourne une valeur `VbMsgBoxResult` : on peut l'ignorer sans problème lorsqu'on veut simplement afficher un message, mais on peut aussi la capturer dans une variable locale et l'utiliser plus tard si nécessaire. Dans ce cas, les parenthèses sont requises et entourent la liste d'arguments.

ARGUMENTS NOMMÉS

Les expressions fournies dans une liste d'arguments peuvent être nommées. C'est utile quand plusieurs valeurs sont transmises à une même procédure (même si, idéalement, les expressions d'arguments restent simples : au besoin, extrayez-les dans des variables locales), ou encore lorsqu'on passe des littéraux dont la signification n'est pas évidente. Par exemple, lorsqu'on passe un littéral booléen `True` ou `False` à une procédure, nommer l'argument peut rendre l'appel beaucoup plus clair sans devoir consulter *IntelliSense* ou la définition de la procédure.

Les arguments nommés sont toujours optionnels; comme ils peuvent alourdir les sites d'appel, utilisez-les avec discernement, au besoin en combinaison avec des continuations de ligne. Lorsque vous répartissez une liste d'arguments nommés sur plusieurs lignes, gardez toujours le nom sur la même ligne que l'expression correspondante : le nom et l'opérateur `:=` font partie de l'argument et ne devraient jamais être séparés.

```
> DoSomething Arg1 := "test", Arg2 := 42, Arg3 _  
    := False
```

Un endroit où les arguments nommés brillent vraiment, c'est quand on passe une variable *par référence* à une procédure qui attribue ensuite à ce paramètre une ou plusieurs valeurs de sortie pour l'appelant. Dans ces cas-là, c'est pratique d'utiliser un préfixe « out » pour signaler que le paramètre est une *sortie* plutôt qu'une *entrée* ; le langage n'a pas de mot-clé `Out` qui pourrait imposer cela au niveau du compilateur, donc c'est à nous d'adopter une convention et de l'utiliser de façon cohérente. `outResult := SomeVariable` indique clairement que `SomeVariable` sera assignée par la procédure appelée, sans avoir besoin d'en lire l'implémentation pour déterminer si (et où) un paramètre `ByRef` pourrait être assigné.

BYREF, BYVAL

Le comportement par défaut (malheureux) de VBA est de passer les paramètres *par référence*. Dans la plupart des cas, cela ne fera aucune différence, mais l'ignorer peut provoquer des bogues subtils et compliquer inutilement le débogage. Passer une variable par référence signifie que la procédure appelée reçoit une référence vers cette variable, plutôt qu'une simple copie de sa valeur. Cela implique que la procédure peut réassigner cette référence et, si le code appelant ne s'y attendait pas, l'état local peut devenir imprévisible ou difficile à diagnostiquer.

Lorsque vous passez une expression entre parenthèses à une procédure qui accepte le paramètre `ByRef`, VBA évalue l'expression et transmet son résultat à la procédure. Autrement dit, vous pouvez entourer un argument de parenthèses lorsque le paramètre est passé `ByRef`, mais que vous ne souhaitez pas passer une référence (par exemple, à une variable) :

DoSomething (SomeVariable)

Ici, `DoSomething` reçoit une référence vers le résultat de l'expression qui évalue la valeur de `SomeVariable`. Sans les parenthèses, la procédure recevrait une référence vers `SomeVariable` elle-même; et si la procédure réassigne la valeur détenue par le paramètre correspondant `SomeParameter`, comme le paramètre est passé `ByRef`, l'adresse de `SomeParameter` est la même que celle de `SomeVariable`. Donc, lorsqu'elle est réassignée dans `DoSomething`, elle l'est aussi partout ailleurs où cette référence est utilisée.

Parce qu'une procédure ne devrait normalement pas pouvoir interférer avec l'état de son appelant, on préférera passer les paramètres `ByVal` dès que possible — surtout lorsqu'on passe des références d'objets. `ByRef` reste utile lorsqu'une procédure doit, de manière évidente et explicite, recevoir un paramètre par référence; par exemple, lorsqu'on écrit une fonction `TrySomething` qui retourne un `Boolean` et qui, conditionnellement, affecte un *paramètre de sortie*. Certains types de données ne peuvent pas être passés par valeur : les tableaux et les structures de type défini par l'utilisateur (UDT) devraient toujours être passés par référence, sauf s'ils sont encapsulés dans un `Variant`.

EN RÉSUMÉ

- Privilégiez le passage de valeurs via des paramètres plutôt que d'augmenter la portée d'une variable au niveau du module, ou de déclarer des variables globales.
- Passez les paramètres `ByVal` dès que possible.
 - Les tableaux et les structures de type défini par l'utilisateur (UDT) ne peuvent pas — et ne devraient pas — être passés par valeur.
 - **Les objets ne sont jamais « passés »**, quel que soit le modificateur : ce qui circule, c'est toujours (même avec `ByVal` : une copie) un *pointeur*. Un pointeur est simplement une valeur entière 32 bits ou 64 bits, selon l'architecture du processus. Passer ce pointeur `ByRef` (explicitement ou non) augmente les occasions d'erreur de programmation.
- Utilisez un modificateur `ByRef` explicite chaque fois que vous passez des paramètres par référence.
- Envisagez d'utiliser un préfixe `out` pour nommer les paramètres de retour passés `ByRef`.
 - Envisagez d'utiliser des arguments nommés pour les paramètres de retour `ByRef` préfixés par `out`.
- Utilisez des parenthèses lorsque vous capturez la valeur de retour d'une procédure, soit dans une variable locale, soit en passant directement le résultat comme argument à une autre procédure.
- N'utilisez pas de parenthèses autour des arguments lorsque vous ignorez la valeur de retour d'une procédure, ou lorsque vous appelez une procédure `Sub` (qui ne retourne rien).



COMMENTAIRES

Les bons commentaires devraient expliquer *pourquoi* quelque chose arrive, ou *pourquoi* quelque chose est fait d'une certaine façon. Les mauvais commentaires sont presque toujours des distractions redondantes qu'il vaut mieux supprimer : laissez le code expliquer le *quoi* et le *comment*. Notez que je distingue les commentaires « dans le code » de la documentation : en VBA, chaque module et chaque membre peut avoir un attribut masqué `VBDescription` qui peut servir à documenter l'objectif du module ou du membre. Avec Rubberduck, vous pouvez maintenir ces attributs



cachés et les afficher sous forme d'annotations de commentaire; c'est un type de commentaire différent, qui doit être traité comme de la documentation pour développeurs : des *docstrings* visibles dans l'outil *Object Browser* (Explorateur d'objets) de l'IDE, et exposées à plusieurs autres endroits dans l'interface Rubberduck (Code Explorer, barre d'outils contextuelle, et éventuellement aussi dans des info-bulles *IntelliSense* personnalisées). Ces commentaires-là peuvent très bien décrire le *quoi*, parce que lorsqu'on les lit, on veut surtout s'assurer qu'on appelle la bonne chose.

Ici, il ne s'agit pas de cette documentation, mais de tout le reste : commentaires à côté d'une déclaration de variable, commentaires insérés au milieu d'une procédure entre deux blocs de code; commentaires qui détournent l'attention du code, ou qui n'ont tout simplement pas leur place. L'art ASCII a sa place dans l'écran de démarrage d'une application console, mais pas au milieu d'une base de code sous forme d'un énorme commentaire envahissant. Les mentions de droits d'auteur et d'information de licence devraient se trouver dans un fichier README et/ou LICENSE (ou, en VBA, dans des métadonnées du document hôte), pas comme un en-tête de commentaire de 40 lignes en haut de chaque module. Versioning, ensembles de changements, numéros de tickets de support : rien de tout cela n'a sa place dans des commentaires; utilisez plutôt un outil de contrôle de versions approprié (git, par exemple).

Nettoyer de vieux commentaires dans un projet hérité peut être difficile : vous lisez le commentaire, puis le code auquel il se rapporte, puis vous relisez le commentaire, et il vous faut quelques précieuses secondes pour réaliser qu'on vous a piégé — le commentaire dit une chose, mais le code en fait une autre. C'est précisément pour cela qu'on veut des commentaires qui expliquent le *pourquoi* bien plus que des commentaires qui décrivent le *quoi*. Un bon commentaire peut sauver la mise lorsqu'il vous rappelle qu'une variable apparemment inutile existe uniquement pour vous permettre de placer un point d'arrêt à un endroit stratégique pendant le débogage; ou lorsqu'il indique qu'on doit chercher le coût *moyen* associé à un SKU afin de calculer correctement des marges, et que c'est pour cela que la recherche se fait dans la table d'inventaire plutôt que dans la table des achats. Les bons commentaires contiennent un savoir précieux qu'on ne peut pas déduire du code seul : laissez le code parler pour lui-même et rester la source de vérité sur ce qu'il fait.

Il n'y a pas mille façons d'écrire des commentaires non distrayants en VBA, mais utiliser le marqueur/mot-clé historique `Rem` plutôt que l'apostrophe simple (`'`) risque de dérouter plus d'un mainteneur. Il vaut aussi mieux éviter les commentaires « poursuivis » sur plusieurs lignes : VBA n'a pas de syntaxe de bloc de commentaires de type `/* ... */`, et ce sont les retours de ligne qui délimitent les instructions (plutôt qu'un terminateur explicite, comme un point-virgule). Un commentaire multi-lignes basé sur des continuations de ligne est inutilement plus difficile à maintenir qu'une simple série de lignes de commentaire. Puisque VBA analyse un commentaire continué comme une seule *ligne logique de code*, un commentaire ne devrait jamais se terminer par un underscore précédé d'un espace : sinon, la ligne suivante (souvent une déclaration) devient une continuation du commentaire au lieu d'être une instruction.

EN RÉSUMÉ

- Utilisez l'apostrophe simple `'` pour indiquer un commentaire.
- Évitez les commentaires avec continuation de ligne; utilisez plutôt une apostrophe en première position sur chaque ligne.
- Envisagez d'ajouter une annotation `@ModuleDescription` au début de chaque module.
- Envisagez d'ajouter une annotation `@Description` pour chaque membre `Public` d'un module.
- Supprimez les commentaires qui décrivent *ce que* fait une instruction; remplacez-les par des commentaires qui expliquent *pourquoi* cette instruction doit faire ce qu'elle fait.



- Supprimez les commentaires qui résument *ce que fait un bloc de code*; remplacez-les par un appel à une nouvelle procédure portant un nom descriptif.
- Évitez d'encombrer un module avec des commentaires « bannière » qui énoncent l'évidence. On sait que ce sont des variables, ou des propriétés, ou des méthodes publiques : pas besoin d'un énorme bandeau de commentaires verts pour nous le rappeler.
- Évitez d'encombrer la portée d'une procédure avec des commentaires « bannière » qui découpent artificiellement ses responsabilités : si une procédure fait trop de choses, découpez-la *et donnez un nom approprié à la nouvelle procédure*.



VARIABLES

Quand on se rappelle que n'importe quelle instruction peut mal tourner et « exploser au visage », on avance prudemment et on gère les erreurs d'exécution. Et ensuite? Si vous enchaînez des appels de membres et fournissez des résultats de fonctions comme arguments, et que ces fonctions reçoivent elles-mêmes des résultats de fonctions en paramètres... vous arriverez peut-être à tout faire en une seule ligne, mais au prix d'un débogage pénible et inefficace si quoi que ce soit se brise n'importe où. Utiliser des variables pour stocker des valeurs ou des références intermédiaires permet de s'arrêter et de valider l'état avant de le consommer. En débogage, cela facilite aussi la revue du résultat de chaque opération. Souvent, on peut éviter de déclencher une erreur d'exécution en capturant une référence ou une valeur dans une variable, puis en abandonnant proprement au lieu de poursuivre avec un état ou des données invalides.

Les variables sont une forme d'abstraction : une variable peut représenter le résultat d'une fonction, ou celui d'une équation complexe à laquelle on donne un nom. Un avantage des variables, c'est qu'elles ont non seulement un nom significatif, mais aussi un *type de données*. Si vous omettez le type, vous obtenez une Variant. Il vaut mieux être explicite quant aux types de données et y prêter attention, *parce que* le langage, lui, s'en soucie souvent très peu — et cela peut devenir problématique si vous n'êtes pas familier avec tous les appels et conversions implicites que VBA peut effectuer à l'exécution. Déclarer un type explicite ne fait pas que signaler votre intention : cela permet aussi une *liaison anticipée* (early binding). Les appels de membres peuvent être validés à la compilation lorsqu'ils sont liés tôt. Sinon, même Option Explicit ne peut pas vous sauver d'une faute de frappe dans un appel de membre à liaison tardive. Utiliser des variables permet donc de garder les choses liées tôt et (idéalement) correctement nommées.

Au niveau du module, gardez les déclarations au minimum : idéalement, seulement de l'état encapsulé Private et des références d'objets WithEvents (principalement dans des modules de classe). Les implications des champs Private et Public dans un module standard peuvent être difficiles à raisonner (pensez à ce que le mot-clé End fait à cet état), et elles ne sont pas nécessairement anodines. La bonne nouvelle, c'est que l'état global « corrompu » n'arrive pratiquement jamais quand on ne dépend pas d'un état global. C'est l'un des avantages de limiter les portées et de passer des paramètres : l'état n'a que très rarement besoin d'être global.

Évitez de dupliquer dans une portée locale ce qui est déjà global. Par exemple, les classes dont l'attribut VBPredeclaredId est défini à True amènent VBA à définir, au niveau du projet, une référence d'objet vers une *instance par défaut* de la classe. Cette *instance par défaut* porte toujours le même nom que la classe elle-même. Et pour rester parfaitement explicites quant à notre intention, on devrait utiliser cet identifiant pour référer à cette instance précise.

Par exemple, dans Excel, `ThisWorkbook` représente toujours le document hôte — le fichier classeur Excel contenant du code VBA. Il n'existe qu'un seul objet `Workbook` pour le document hôte et, *dans ce document*, tout code qui veut référer à cet objet devrait le faire via l'identifiant `ThisWorkbook`. Stocker cette référence dans une variable locale portant un autre nom ne sert à rien : cela ajoute une couche *d'indirection* qui masque ce qui se passe réellement et duplique une référence d'objet déjà globale.

Les variables devraient être déclarées dans la portée d'une procédure *au moment où leur présence devient pertinente*. Cela peut entrer en conflit avec votre style, mais imaginez si l'introduction du *Seigneur des anneaux* ou de *Game of Thrones* vous donnait d'emblée le nom et l'arbre généalogique de tous les personnages qui apparaîtront plus tard... puis que cette information disparaissait du récit : vous perdriez rapidement le fil et vous vous demanderiez constamment qui est qui. À l'inverse, ces informations sont introduites progressivement, ce qui permet de se concentrer sur l'intrigue tout en assimilant les nouveaux éléments.

Saturer l'esprit du lecteur d'informations pas encore pertinentes (si elles le deviennent un jour!) en déclarant toutes les variables locales en un gros *bloc* au début d'une procédure produit le même effet : *bombarder* quelqu'un d'informations n'est généralement pas une approche judicieuse. D'expérience, déclarer toutes les variables en haut augmente fortement le risque de laisser en place un nombre parfois surprenant de déclarations inutilisées.

Déclarer les variables locales au moment où vous en avez besoin, et au moment où leur affectation devient nécessaire, place la déclaration dans son contexte d'utilisation. Cela peut réduire le besoin de commentaires explicatifs et règle tous les problèmes évoqués ci-dessus. Ce n'est clairement pas un argument du type « parce que tout le monde le fait » (même si, dans la plupart des autres langages, on déclare effectivement les choses au fur et à mesure qu'elles sont nécessaires).

Cela implique aussi de déclarer les variables locales *séparément*, avec une instruction `Dim` par variable, suivie de l'affectation de cette variable. J'aime garder ces paires « déclaration + affectation » ensemble, avec un peu d'espace au-dessus et au-dessous : si une procédure ne fait qu'une chose et le fait bien, elle n'a souvent, naturellement, pas besoin de tant de variables que ça.

EN RÉSUMÉ

- Déclarez toutes les variables, toujours. Option `Explicit` devrait toujours être activée.
- Déclarez toujours un type de données explicite. Si vous voulez dire `As Variant`, alors écrivez `As Variant`.
- Envisagez d'utiliser un `Variant` pour passer des tableaux entre des portées, plutôt que des tableaux typés (par exemple `String()`).
 - Mettez ces identifiants au pluriel : cela signale une pluralité d'éléments de façon bien plus élégante que la *Pirate Notation* (les préfixes `arr*`).
- Évitez les champs `Public` dans les modules de classe; encapsulez-les plutôt avec une `Property`.
- Envisagez d'utiliser une structure d'appui basée sur un `Private Type` pour les champs d'appui des propriétés d'une classe : cela élimine le besoin d'un schéma de préfixes, permet de nommer une propriété exactement comme son champ d'appui correspondant, et nettoie la fenêtre d'outils *Locals* en regroupant les champs sous une seule variable de module.
- Limitez autant que possible la portée des variables. Préférez passer des paramètres et conserver les valeurs en portée locale plutôt que de promouvoir une variable à une portée plus large.

- Déclarez les variables là où vous les utilisez, *au moment où vous en avez besoin*. Vous ne devriez jamais avoir à faire défiler le code pour retrouver la déclaration d'une variable que vous êtes en train de lire.



LIAISON TARDIVE (LATE BINDING)

Déclarer et utiliser des variables avec un type de données spécifique — en particulier des *types d'objets* — permet de rester de façon cohérente en *liaison anticipée* (*early binding*) : le compilateur et les outils d'analyse statique disposent alors de la meilleure compréhension possible du code. La *liaison tardive* (l'inverse : la résolution des types et des membres est reportée à l'exécution) a peu à voir avec `CreateObject` ou le fait qu'une bibliothèque soit référencée. En réalité, la *liaison tardive* survient aussi implicitement, assez facilement, et beaucoup plus souvent que ce qui serait sain. Efforcez-vous de rester autant que possible en liaison anticipée : lorsque le compilateur et *IntelliSense* ne savent pas ce que vous faites, vous êtes seul — et même `Option Explicit` ne pourra pas vous sauver d'une faute de frappe (et de l'erreur 438).

Vous savez que vous faites un appel de membre à liaison tardive lorsque vous tapez l'opérateur *d'accès aux membres* (le point) et que rien ne se produit. En liaison anticipée, cela déclenche normalement *IntelliSense* et l'affichage d'une liste de complétion dans l'éditeur. Sans information de type disponible à la compilation, l'éditeur ne peut pas vous assister, et le compilateur acceptera à peu près n'importe quel identifiant comme appel de membre, en reportant sa résolution à l'exécution. Quand cela arrive, il vaut mieux s'arrêter, déterminer quelle interface vous souhaitez réellement utiliser (et vous assurer que l'objet d'exécution l'implémente, sinon vous obtiendrez une erreur d'exécution de type *Type mismatch*), déclarer une variable locale de ce type, l'assigner à l'expression à laquelle vous alliez chaîner l'appel de membre, puis faire l'appel de membre en liaison anticipée sur cette variable locale.

Par exemple, lorsque vous récupérez un objet `Worksheet` à partir de la collection `Sheets` du modèle objet Excel, ce que vous obtenez est un `Object` qui peut être un `Worksheet`, un `Chart`, ou encore une demi-douzaine d'autres types historiques de « feuilles ». La plupart du temps, c'est effectivement un `Worksheet`, mais le modèle objet ne peut pas le présumer — et votre code ne devrait pas le faire non plus. Tout appel de membre chaîné à ce résultat sera à liaison tardive et peut échouer avec l'erreur 438 si votre hypothèse est fausse. C'est l'une des raisons pour lesquelles on préfère la collection `Workbook.Worksheets` à `Workbook.Sheets` lorsqu'on veut récupérer une instance de `Worksheet` : les deux retournent un `Object` et doivent être « castées » en `Worksheet`, mais une seule garantit qu'on obtient bien un objet `Worksheet` (ou une autre collection `Sheets` si on lui a fourni un tableau de noms de feuilles).

L'opérateur *d'accès dictionnaire* (aussi appelé *bang*) ! est un autre exemple de fonctionnalité qui semble avoir été ajoutée pour faciliter l'écriture du code, avec peu ou pas de considération pour sa lecture. Et comme le symbole ! est syntaxiquement ambigu (c'est aussi le caractère d'indication de type pour `Single`), il peut ralentir sensiblement l'analyse, parce que la grammaire ambiguë nécessite davantage d'itérations pour déterminer la règle correcte.

L'opérateur fonctionne en interrogeant l'objet pour trouver un *membre par défaut* qui accepte un seul paramètre `String` ; l'« identifiant » qui suit l'opérateur est (sans surprise?) l'argument `String` passé à ce membre par défaut — d'où le besoin fréquent de crochets lorsque les noms peuvent contenir des espaces. Mais ce n'est pas toute l'histoire : dans bien des cas, ce que le membre par défaut retourne, c'est une référence vers un autre objet qui, *lui aussi*, possède un membre par défaut... et c'est *cela* que l'expression d'accès dictionnaire finit par résoudre. Au final, on obtient une « boîte noire » complète quant aux types de classes impliqués : cet opérateur déclenche une magie implicite et met beaucoup de mécanismes en mouvement simplement pour éviter d'écrire `Recordset.Fields("FieldName").Value`. Pourtant, cela pourrait très bien devenir un appel de fonction



GetFieldValue(Recordset, "FieldName") qui explicite l'intention *une seule fois* — et on peut même « commenter » l'incantation équivalente Recordset!FieldName à côté, au besoin.

EN RÉSUMÉ

- Évitez d'appeler des membres sur un **Object** ou un **Variant**. Si un type est disponible à la compilation et utilisable pour cet objet, assignez la référence à une variable locale de ce type (avec **Set**), puis faites l'appel de membre en liaison anticipée sur cette variable locale.
 - Prendre un objet qui présente une interface et l'assigner à un autre type s'appelle un « cast ». Si l'objet ne supporte pas (n'implémente pas) la nouvelle interface, le cast / la conversion échoue.
- Bien sûr, la *liaison tardive explicite* est acceptable (déclarer **As Object**, et créer des instances de classes provenant de bibliothèques non référencées avec **CreateObject** plutôt qu'avec l'opérateur **New**). La liaison tardive est très utile et a de nombreux cas d'usage légitimes, mais généralement pas lorsque le type d'objet est accessible à la compilation via une référence de bibliothèque.

Évitez l'opérateur *d'accès dictionnaire* (alias *bang*) ! : par définition, il est à liaison tardive, et il fait en sorte qu'un simple littéral **String** se lise comme un nom de membre; de plus, tout appel de membre chaîné à ce résultat sera inévitablement à liaison tardive lui aussi. Rubberduck peut analyser et résoudre ces expressions, mais elles sont beaucoup plus coûteuses à traiter que des appels de méthodes standards.



EXPLICITÉ

Rubberduck vous encourage à être systématiquement explicite sur certaines choses, et systématiquement implicite sur d'autres. Par exemple, le nom d'identifiant optionnel après le mot-clé **Next** est considéré comme redondant, parce qu'un corps de boucle qui ne contient qu'un seul appel de procédure paramétré ne peut pas s'étendre sur suffisamment de lignes pour que vous perdiez le fil de ce que vous itérez. En revanche, un modificateur explicite **ByRef** ou **Public**, même s'il est parfois redondant, signale une intention bien plus claire que son absence.

Mais l'*explicitité* va au-delà des jetons optionnels dans la syntaxe : c'est aussi éviter les références implicites. Par exemple, si vous automatisez Word depuis Excel et que vous utilisez **Word.Documents** au lieu de récupérer cette collection à partir d'une référence précise vers un objet **Word.Application**, vous ouvrez la porte à des comportements inattendus.

De nombreuses bibliothèques exposent des classes qui ont un *membre par défaut* sans paramètre; lorsque vous voulez référer à ce membre, faites-le explicitement. Par exemple, **Application.Name** est implicite dans **Debug.Print Application**, et il ne devrait pas l'être, parce qu'il est impossible de deviner l'appel implicite à partir du code seulement : le lecteur *doit* savoir que cette classe d'objet *Application* se réduit implicitement à sa propriété *Name*. Le seul moyen de le savoir est d'aller le vérifier dans l'*Object Browser* (Explorateur d'objets), où l'on peut afficher les membres masqués; le membre par défaut d'une classe y est indiqué par un petit point bleu superposé à son icône.

EN RÉSUMÉ

- Utilisez des modificateurs explicites partout (**Public/Private**, **ByRef/ByVal**).
- Déclarez un type de données explicite, même (et surtout!) lorsque c'est **Variant**.



- Évitez les *qualificateurs implicites* dans tous les appels de membres : dans Excel, surveillez les références implicites à ActiveSheet, les références implicites à ActiveWorkbook, les références implicites à la feuille conteneur et les références implicites au classeur conteneur : ce sont une source extrêmement fréquente de bogues.
- Appelez explicitement les *membres par défaut* sans paramètre.
 - Note : certains modèles objet définissent un membre par défaut *masqué* (p. ex. Range.[Default]) qui redirige vers un autre membre selon sa paramétrisation. Dans ces cas, il vaut mieux appeler directement le membre visé; par exemple, utilisez Range.Value lorsque c'est approprié. Le membre masqué [Default], lui, gagnerait généralement à ne pas être appelé du tout, pour des raisons de lisibilité et de performance.
- Appelez implicitement les membres par défaut paramétrés lorsqu'il s'agit d'indexeurs qui récupèrent un élément d'une collection d'objets, par exemple la propriété Item d'une Collection. Les appeler *explicitement* n'est pas problématique, mais peut être considéré comme inutilement verbeux.
- Call n'a pas besoin de faire partie du vocabulaire de votre code lorsque vous utilisez des noms de procédures expressifs et descriptifs qui *suggèrent* clairement l'action.
- Envisagez de qualifier explicitement les appels aux membres de modules standards avec le nom du projet (et du module), y compris pour les bibliothèques standards et référencées, surtout dans les projets VBA qui référencent plusieurs modèles objet.



GESTION DES ERREURS

Chaque procédure devrait-elle systématiquement avoir une routine locale de gestion des erreurs? Pour moi, la réponse est un « non » sans équivoque, parce que j'aime contrôler quels erreurs peuvent *remonter* des méthodes à l'exécution, et parce qu'une routine de gestion des erreurs qui ne fait que relayer l'erreur à l'appelant est redondante.

Pour comprendre la gestion des erreurs en VBA, il faut d'abord comprendre comment VBA se comporte à l'exécution.

CHEMIN NOMINAL

Quand VBA entre dans la portée d'une procédure, il n'existe qu'un seul « chemin nominal ». Une instruction On Error indique à VBA de sauter vers une routine locale de gestion des erreurs, au lieu de sortir complètement de la portée de la procédure.

```
Public Sub DoSomething()
    On Error GoTo CleanFail
    ' ...chemin nominal ici
CleanExit:
    ' ...code de nettoyage ici
    Exit Sub
CleanFail:
    ' ...chemin d'erreur ici
    Resume CleanExit ' commenter pour déboguer
    Stop
    Resume
End Sub
```

Il est important de séparer clairement le code qui est autorisé à s'exécuter pendant un état de récupération après erreur, du code qui s'exécute normalement — le « chemin nominal » ne devrait jamais être accessible pendant qu'une erreur est en cours de traitement.

Mêler chemin nominal et chemin d'erreur vous fera vous demander ce qui se passe lorsqu'une erreur d'exécution survient malgré l'instruction On Error.

CHEMIN D'ERREUR

Lorsqu'une instruction On Error est active et qu'une erreur survient dans le chemin nominal, l'exécution entre dans la routine locale de gestion des erreurs, et VBA passe en *état de récupération* : l'objet Err contient des informations sur l'erreur d'exécution, et Resume peut vous renvoyer directement à l'instruction qui a déclenché l'erreur.

Ce que vous voulez faire ici, c'est récupérer l'état si possible et reprendre l'exécution normale. Si vous ne pouvez pas récupérer à partir de l'état actuel, utilisez une instruction Resume pour réinitialiser explicitement l'état d'erreur et sauter à une étiquette à la fin du chemin nominal, qui s'exécute qu'il y ait eu une erreur ou non : ainsi, l'exécution sort toujours au même endroit, et vous savez que quelque chose ne va pas lorsque ce n'est pas le cas. Évitez de lever des erreurs pendant que vous êtes en état d'erreur, sauf si votre intention est de « relancer » (rethrow) une erreur dans la pile d'appels pour que l'appelant la gère :

En relançant le même numéro d'erreur, on ne perd pas la description présente dans l'objet global Err :

```
Public Sub DoSomething()
    On Error GoTo CleanFail
    ' ...chemin nominal ici
CleanExit:
    ' ...code de nettoyage ici
    Exit Sub
CleanFail:
    ' ...chemin d'erreur ici
    With Err
        Select Case .Number
            Case Errors.ERRSomeCustomError
                ' ...
                Resume CleanExit
            Case Else
                .Raise .Number ' relancer
        End Select
    End With
    Stop
    Resume
End Sub
```

Ce qui est crucial, c'est qu'à aucun moment, pendant l'exécution d'une routine de gestion d'erreurs, l'exécution ne revienne dans le *chemin nominal* sans avoir d'abord réinitialisé l'état d'erreur. La manière la plus sûre d'y parvenir est de s'assurer que le chemin d'erreur rencontre toujours une instruction Resume.

L'étiquette CleanExit peut ne contenir qu'une instruction Exit, mais son rôle est littéralement de faire sortir la procédure *proprement*, sans entrer dans la section qui est censée s'exécuter pendant qu'on est en état d'erreur. Par exemple, si la portée possède une connexion à une base de données, c'est un bon endroit pour la nettoyer.

EN RÉSUMÉ

- Séparez clairement le *chemin nominal* du *chemin d'erreur* : une fois en gestion d'erreurs, n'exécutez plus de code du chemin nominal sans avoir réinitialisé l'état d'erreur.
- Utilisez une structure de sortie cohérente, par exemple CleanExit / CleanFail, et terminez le chemin d'erreur par un Resume vers CleanExit (ou autre point de sortie).
- Évitez les routines qui ne font que relayer l'erreur à l'appelant : si vous ne gérez rien localement, laissez l'erreur remonter.
- N'utilisez pas la gestion d'erreurs comme mécanisme de contrôle du flux normal : validez les hypothèses et les entrées autant que possible avant que ça « explose ».
- Quand vous devez relancer une erreur, relancez le même numéro (p. ex. .Raise .Number) pour conserver la description portée par Err.



PROGRAMMATION STRUCTURÉE (PROCÉDURALE)

Avant la programmation procédurale structurée, BASIC n'avait pas vraiment de *modules* : on écrivait un script et on prenait soin de *numéroter chaque ligne* en laissant suffisamment d'espace (souvent en incréments de 10) pour permettre au « module » de grossir, puis des instructions GOTO et GOSUB/RETURN nous envoyaient vers et depuis des *sous-routines* à l'intérieur du script. Quand elle est apparue, la programmation procédurale a représenté un changement de paradigme : on pouvait enfin organiser un projet d'envergure en modules spécialisés, et structurer ces modules avec des portées de procédures clairement nommées, plutôt qu'en jouant avec des trous dans la numérotation des lignes. Avec l'arrivée de la *gestion structurée des erreurs* et de l'objet Err, la numérotation des lignes est devenue complètement archaïque et inutile dans du code moderne.

Une grande partie du vieux code BASIC fonctionne encore (avec des ajustements mineurs) en VBA : jusqu'à .NET, le langage a évolué sans briser grand-chose sur le plan de la compatibilité rétroactive. Résultat : il existe de nombreux mots-clés et constructions « paléo » dans le langage pour préserver cette compatibilité; leur présence ne signifie pas qu'on devrait les utiliser. Il y a une raison pour laquelle ils ont été remplacés par d'autres mots-clés et constructions. While...Wend est un bon exemple : il n'existe aucune raison rationnelle de le préférer à la construction plus standard et flexible Do While...Loop. Les instructions Error\$ vous donnent le message d'erreur courant, mais en code moderne vous voulez plutôt utiliser l'objet Err. Et Global ne signifie rien d'autre que Public en VBA. Personne n'écrit encore On Local Error aujourd'hui, et pourtant, comme le reste, tout cela est supporté et fonctionne comme prévu.

Cela dit, la *programmation procédurale* est un paradigme parfaitement adapté pour écrire du code VBA : vous consommerez des objets (parfois sans vous en rendre compte!) bien plus souvent que vous n'en écrirez. Et c'est très bien. Ce qui compte, c'est de garder le code *structuré*, pas seulement « procédural ». Mettez des macros distinctes dans des modules distincts, avec une procédure Public de très haut niveau d'abstraction au sommet, qui appelle des procédures Private paramétrées de plus en plus détaillées. Avoir des niveaux d'abstraction cohérents est un art : vous voulez manipuler des cellules, des plages et l'API Win32 au niveau d'abstraction le plus bas; les niveaux supérieurs mettent des mots sur ce qui se passe, pour que vous lisiez « préparer l'en-tête du rapport » plutôt que « écrire une série de valeurs dans une série de cellules ».

Les principes classiques de programmation s'appliquent pleinement en procédural : *Don't Repeat Yourself* (DRY) et *Keep It Simple, Stupid* (KISS), notamment. Vous voudrez donc structurer vos modules comme des histoires qui se déroulent au fur et à mesure qu'on descend dans des procédures Private de plus bas niveau d'abstraction. En pratique, cela implique que votre module typique ne contient qu'une seule procédure publique au sommet, qui incarne une macro à son plus haut niveau d'abstraction.

Inévitablement, vous allez écrire une autre macro/un autre module, puis réaliser qu'une bonne partie des opérations de bas niveau se ressemble beaucoup. Et là, vous copiez-collez un bloc de code dans un nouveau module? Non. C'est plutôt le moment d'extraire une procédure dans son propre module avec Option Private Module, de rendre la procédure extraite Public, puis de l'invoquer depuis les deux endroits.

Avec le temps, vous constaterez que les procédures d'aide qui méritent ce traitement sont souvent de petites opérations communes qui gagnent à être nommées. Par exemple, si vous calculez une marge brute à plusieurs endroits, il peut être utile d'extraire une fonction qui prend un coût et un prix de vente en paramètres, valide les entrées, et retourne le résultat (ou une valeur sentinelle lorsque le prix de vente est nul).

La validation des entrées devrait être systématique : on ne veut jamais diviser par zéro. Plutôt que d'en faire la responsabilité de l'appelant, une fonction devrait vérifier que l'opération peut être exécutée de façon sécuritaire *avant* de provoquer une erreur d'exécution évitable. Parfois, des entrées invalides peuvent être gérées directement : par

exemple, une fonction `CalculateGrossMargin` sait qu'elle ne peut pas calculer un résultat sans prix, mais il peut aussi être logique de retourner -100 % lorsque vous donnez quelque chose gratuitement alors que son coût est non nul. En revanche, un coût *négalif* n'a pas de sens : la fonction devrait refuser de traiter cette valeur. Lever une erreur d'exécution personnalisée avec un message utile est alors la meilleure option, parce que c'est un problème *exceptionnel* du côté de l'appelant.

Être *défensif* à l'égard des entrées — c'est-à-dire ne faire confiance à aucune d'entre elles quant aux valeurs attendues (ou même à une plage de valeurs) — rend votre code beaucoup plus robuste, et quand quelque chose va mal, vous le savez tout de suite. Imaginez qu'on ait des données erronées, comme un coût négatif, puis que le pourcentage de marge brute soit réutilisé plus loin pour produire une projection : sans validation, vous découvrirez le problème à la fin du processus, lorsque le rapport sort des chiffres incohérents. Avec validation, la macro peut s'arrêter dès qu'elle rencontre la donnée invalide, et présenter un message clair à l'utilisateur.

Parfois, les données viennent directement de l'utilisateur : avec des entrées utilisateur, il faut être encore plus défensif, et coder comme si l'utilisateur faisait tout ce qu'il peut pour faire tomber votre code. Peut-être qu'une chaîne est vide alors qu'elle ne devrait pas l'être; peut-être qu'un code de devise n'existe pas dans votre table de référence; peut-être qu'il y a une faute de frappe et que l'utilisateur a saisi 456 au lieu de 123; peut-être qu'une date est dans le futur et ne devrait pas l'être; peut-être qu'un séparateur décimal diffère. Votre code ne peut pas supposer que les nombres auront la même forme pour tous les utilisateurs. Ou encore, une boîte de dialogue peut simplement être annulée. Il y a mille façons pour lesquelles les entrées utilisateur peuvent mal tourner; ne pas les valider, ou supposer qu'elles sont correctes, fera inévitablement remonter des bogues — et jamais au bon moment.

Vous voudrez que votre procédure de plus haut niveau gère les erreurs qui n'auraient pas été gérées par les niveaux inférieurs, mais vous voudrez traiter différemment les *erreurs de programmation* et les *erreurs utilisateur*. Il est acceptable qu'une erreur de programmation interrompe l'exécution avec un message natif (même cryptique), mais dans la plupart des cas vous voudrez afficher à vos utilisateurs des messages beaucoup plus clairs et plus conviviaux lorsqu'il s'agit d'un problème qu'ils peuvent corriger. Par exemple : « le SKU 1234 n'a pas de prix de vente », plutôt qu'un banal « division by zero » survenant plus tard pendant l'exécution.

Le code de bas niveau est celui où les erreurs inattendues sont les plus probables, parce qu'il se trouve presque toujours à la frontière d'un autre système : quelque chose d'aussi simple que lire la valeur d'une cellule dans une variable locale peut échouer de plusieurs façons si vous faites des hypothèses (faites autant confiance aux cellules qu'aux entrées utilisateur — c'est-à-dire *pas du tout*!). Travailler avec des fichiers, un système de fichiers, un réseau, n'importe quel type de connexion : tout cela dépend de composants sur lesquels votre code n'a aucun contrôle, et qui devraient pouvoir échouer proprement.

EN RÉSUMÉ

- Une macro / un script par module. Mettez-la dans un module standard plutôt que dans le code-behind d'une feuille de calcul.
- Public en premier, suivi de procédures Private paramétrées, avec des niveaux d'abstraction décroissants : le haut se lit comme un résumé, le bas comme une suite d'opérations petites, spécifiques... et plutôt ennuyeuses.
 - Vous savez que c'est bien fait quand vous introduisez une deuxième macro/un deuxième module, et que vous pouvez extraire les petites opérations de bas niveau dans des membres Public d'un module utilitaire, puis les réutiliser.
- Ne vous répétez pas (DRY).

- Envisagez de passer l'état pertinent à une autre procédure en entrant dans un bloc de code. Le code est plus simple et plus facile à suivre lorsque le corps d'une boucle ou d'un bloc conditionnel est déplacé dans sa propre portée.
- Évitez d'utiliser la gestion d'erreurs pour contrôler le flux d'exécution : la meilleure gestion d'erreurs, c'est souvent de ne pas en avoir, parce que les hypothèses sont vérifiées et les choses sont validées. Par exemple, au lieu d'ouvrir un fichier à partir d'un paramètre, vérifiez d'abord que le fichier existe plutôt que de gérer une erreur *file not found*... tout en continuant à gérer les erreurs pour les situations *exceptional* qui peuvent survenir pendant l'accès au fichier.
- Quand il n'est pas possible d'éviter la gestion d'erreurs, envisagez d'extraire une fonction Boolean qui « avale » l'erreur attendue et retourne False en cas d'échec, afin de simplifier la logique.
- Gérez les erreurs autour de toute E/S de fichiers et de réseau.
- Ne faites jamais confiance aux entrées utilisateur : ne supposez pas qu'elles sont valides ni formatées comme vous l'attendez.

PROGRAMMATION ORIENTÉE OBJET

En VBA/VB6, on peut aller plus loin que le simple scripting et appliquer des principes de programmation orientée objet (POO) — encore plus pertinent en VB6 et dans les projets VBA de plus grande taille. Pendant des années, on nous a répété que VBA/VB6 ne peut pas faire de « vraie » POO parce que le langage ne supporte pas l'héritage. La réalité, c'est qu'il y a *beaucoup* plus que l'héritage dans la POO; et si vous voulez apprendre et appliquer des principes POO dans votre code VBA/VB6, vous le pouvez tout à fait — et, honnêtement, vous le devriez. Rubberduck vous aidera à y arriver.

Les principes qui s'appliquent à la programmation procédurale restent valables en POO, mais la POO implique surtout des classes et des objets. On peut grossièrement distinguer deux grands types d'objets : des objets de *données*, et des objets de *services*.

Les classes de données servent à encapsuler des données. En Java, on parle de *beans*; en C#, on parle de POCO (*Plain Old CLR Objects*); et vous avez peut-être déjà vu le terme DTO (*Data Transfer Object*). C'est la même idée : des objets qui ne font essentiellement *rien* d'autre que contenir des données. Ce sont les données, et ils ressemblent souvent à un module simple qui expose un ensemble de propriétés.

L'autre type de classe encapsule un processus. Une classe de service *avec état* peut encapsuler ses propres données/son propre état, et peut même exposer des propriétés pour cela; mais elle expose aussi des *méthodes* qui *font quelque chose* avec ces données. Une telle classe de service peut dépendre d'autres services, qui ont eux-mêmes leurs propres dépendances.

En POO, lorsqu'une classe crée une instance d'une autre classe, ou lorsqu'elle consomme une classe « concrète » particulière, on dit qu'elle y est *couplée*. Le couplage n'est pas intrinsèquement mauvais : en général, on veut que des classes étroitement liées fonctionnent ensemble. Mais quand on a besoin de *découpler* une classe d'une autre, la POO offre des outils pour améliorer les abstractions : on peut extraire une *interface* à partir d'une classe concrète, et cette interface devient une abstraction sur laquelle on peut s'appuyer. Du code qui consomme une interface abstraite ne devrait jamais avoir à se soucier du type concret d'exécution de l'objet.

À un moment donné, il faut tout de même savoir quelles implémentations concrètes seront utilisées. Avec l'*injection de dépendances* (*dependency injection*, DI), on peut choisir une implémentation au démarrage; puis, dans (ou près de) la procédure *point d'entrée*, on crée les instances dont on a besoin et on les injecte.

L'injection de dépendances fonctionne bien avec la *composition*, qui survient lorsqu'un objet en encapsule un autre : en VBA, les classes peuvent avoir des relations « has-a », mais pas « is-a » (l'héritage de classes n'est pas supporté). Les classes VBA n'ont pas de constructeurs paramétrés, alors comment injecter nos dépendances?

Même si l'*injection par constructeur* est la méthode privilégiée dans d'autres langages, plusieurs autres techniques DI peuvent être implémentées en VBA — à commencer par l'*injection par propriété*, combinée à des *méthodes factory* qui tirent parti de l'*instance par défaut* des modules de classe. Si vous n'êtes pas familier avec le vocabulaire DI : l'injection de dépendances, c'est essentiellement une façon plus « chic » de dire « passer des paramètres ». Une « dépendance » est un objet dont votre classe de service a besoin pour faire son travail. Par exemple, une classe de service qui expose des méthodes pour enregistrer quelque chose dans une base de données dépendra de types de la bibliothèque ADODB (Connection, Command, Recordset). Une classe qui dépend de ce service serait donc, indirectement, couplée à ADODB si elle dépendait directement de ces types. Pour découpler ADODB du reste du code, on fait en sorte que le service « base de données » implémente une interface abstraite, et le code qui a besoin du service peut désormais dépendre de cette abstraction. Le code d'initialisation, lui, est responsable de créer le service ADODB « concret » et d'injecter cette instance dans la classe de service.

Tout ça... pour finir par se connecter à une base de données quand même? La différence, c'est qu'il devient alors très facile de remplacer l'implémentation : par exemple, faire en sorte que le service écrive plutôt dans un fichier texte, simplement en injectant une autre implémentation de la même interface abstraite. Des tests pourraient alors vérifier que la classe de service fait son travail *sans avoir besoin d'accéder à une base de données*. Si votre interface d'accès aux données implique des suppressions d'enregistrements, pouvoir tester la totalité des fonctionnalités de l'application sans se connecter à la base de données a de sérieux avantages.

L'injection de dépendances facilite aussi l'application des principes SOLID, au cœur de la programmation orientée objet, et indépendants du langage. Séparer une classe de ses dépendances aide à respecter le *Single Responsibility*. Dépendre d'une interface abstraite satisfait le *Dependency Inversion*, rend plus simple le *Open/Closed*, et si vous évitez de traiter à part une implémentation particulière (c.-à-d. qu'une classe de service n'a pas le droit de « connaître » des détails sur une implémentation concrète de ses dépendances), alors vous couvrez aussi le *Liskov Substitution*. Gardez vos interfaces aussi claires et simples que possible (idéalement, une interface abstraite n'a qu'un seul membre), et vous respectez l'*Interface Segregation*.

Une DI « propre » se fait à un seul endroit dans le code : cet endroit est appelé le *composition root*, parce que c'est là que le *graphe de dépendances* de l'application est construit. Pourquoi un graphe? Parce que si vous avez un objet « racine » (par exemple une classe App), et que cet objet a ses dépendances, et que chacune de ces dépendances a aussi ses propres dépendances, etc., vous obtenez un *graphe d'objets* — un peu comme le modèle objet d'Excel.

Le *composition root* devrait être aussi proche que possible du point d'entrée de l'application. Mais en VBA, le point d'entrée n'est pas toujours évident, ni toujours au même endroit. Si vous écrivez un add-in, le meilleur endroit est probablement le gestionnaire de l'événement *WorkbookOpen* dans le module *ThisWorkbook* : le graphe d'objets peut alors vivre au niveau instance dans *ThisWorkbook*, et rester « vivant » tant que l'add-in est chargé dans Excel. Dans un classeur avec macros, la même approche peut convenir (le graphe vit tant que le classeur reste ouvert), mais le point d'entrée peut aussi être un gestionnaire *Click*, ou une macro/procédure publique attachée à une forme ou à un bouton — et il peut alors y avoir plusieurs points d'entrée. L'endroit idéal dans *votre* projet dépend surtout de la manière dont vous voulez gérer le cycle de vie de ces objets. La manière la plus simple est de gérer *WorkbookOpen*

et de laisser l'application vivre tant que le fichier hôte est ouvert, sans vous soucier de nettoyer quoi que ce soit. Si cela pose problème, vous devrez peut-être gérer `BeforeClose` et démanteler correctement l'application.

Parfois, on ne veut pas nécessairement créer l'instance de l'application au démarrage, parce que notre « application » est plutôt un ensemble de macros pas vraiment liées entre elles, que l'utilisateur peut exécuter ou non (aucune, une seule, ou plusieurs) — et on ne veut pas instancier des objets qu'on n'utilisera pas.

D'autres fois, on ne peut tout simplement pas créer une dépendance au démarrage, soit parce qu'il manque une information que l'utilisateur doit fournir à l'exécution, soit parce qu'on ne veut pas qu'une instance particulière vive aussi longtemps. Dans ces cas où il faut *différer* la création d'une dépendance, on peut injecter une *factory abstraite* à la place, puis n'invoquer sa méthode *Create* (en lui passant les valeurs d'exécution requises comme arguments) qu'au moment où l'on est prêt à créer et consommer l'objet. Notez que la classe qui crée un objet à partir d'une factory devrait aussi être responsable de détruire correctement cet objet (p. ex. fermer des fichiers, libérer des connexions).

EN RÉSUMÉ

- Appliquez les bonnes pratiques POO standard : ce sont des concepts généraux, indépendants du langage, et ils se moquent des capacités exactes de VBA/VB6 :
 - **Single Responsibility Principle** — chaque abstraction ne devrait être responsable que d'une seule chose.
 - **Open/Closed Principle** — écrivez du code qui n'a pas besoin de changer, sauf si le but même de l'abstraction doit changer.
 - **Liskov Substitution Principle** — le code devrait emprunter les mêmes chemins d'exécution, quelle que soit l'implémentation concrète d'une abstraction donnée.
 - **Interface Segregation Principle** — gardez les interfaces petites et spécialisées; évitez un design qui nécessite constamment d'ajouter de nouveaux membres à une interface.
 - **Dependency Inversion Principle** — dépendez d'abstractions, pas d'implémentations concrètes.
- Privilégiez la *composition* lorsque l'*héritage* serait normalement requis.
- Vous ne pouvez pas avoir de *constructeurs paramétrés*, mais vous pouvez tout de même utiliser l'*injection par propriété* dans des méthodes factory pour injecter des dépendances au niveau instance.
- Utilisez l'*injection par méthode* pour injecter des dépendances au niveau d'une méthode.
 - Évitez d'instancier des dépendances « sur place » avec `New` : cela *couple* une classe à une autre, ce qui nuit à la testabilité; *injectez* plutôt les dépendances.
 - Utilisez le mot-clé `New` dans votre *composition root*, le plus près possible d'un *point d'entrée*.
 - Le gestionnaire d'événement `WorkbookOpen` (Excel) est un point d'entrée possible.
 - Les macros (procédures `Sub` invoquées depuis l'extérieur du code) sont aussi des points d'entrée valides.
 - Abandonnez l'idée qu'un module doit contrôler chacune de ses dépendances : laissez *quelque chose d'autre* gérer la création (et la libération) de ces objets.

LIGNES DIRECTRICES -

- Injectez une *factory abstraite* lorsqu'une dépendance ne peut pas (ou ne devrait pas) être créée au *composition root*, par exemple si vous devez vous connecter à une base de données et que vous voulez garder l'objet de connexion vivant le moins longtemps possible.
- Gardez l'*instance par défaut* d'une classe aussi *sans état* que possible. Protégez activement contre les usages accidentels (p. ex. en levant une erreur d'exécution au besoin).
- Utilisez des modules standards plutôt qu'une « classe utilitaire » annotée `@PredeclaredId`, qui n'est jamais instanciée explicitement ni utilisée comme un véritable objet.



INTERFACES UTILISATEUR

Le code d'interface utilisateur est *par nature* orienté objet. Un UserForm devrait donc être traité comme l'objet qu'il est. Les responsabilités d'une interface utilisateur sont simples : afficher et recueillir des données à destination/en provenance de l'utilisateur, et/ou offrir un moyen d'exécuter des *commandes* (qui consomment ces données ou les manipulent).

En VBA, on peut créer des interfaces utilisateur personnalisées au moyen d'un module de classe UserForm. Ce type de module vient avec un attribut VBPredeclaredId défini à True, ce qui signifie que — comme pour ThisWorkbook et les modules de feuilles — on dispose d'un identifiant de portée globale (portant le nom de la classe du formulaire) qui réfère à l'*instance par défaut* de la classe. Cependant, un classeur et une feuille de calcul sont des *modules de document* appartenant à l'application hôte (Excel) : il n'y a pas vraiment de scénario où cela « peut mal tourner ». Un formulaire, lui, appartient au projet VBA, et vous pouvez créer autant d'instances que nécessaire. Cela devient facilement problématique, parce qu'un code qui réfère à l'*instance par défaut* agit sur *cette instance précise*, qui n'est pas forcément celle qui est affichée, selon la façon dont vous utilisez votre formulaire. Pour cette raison, on devrait traiter l'*instance par défaut* d'un formulaire comme on traite l'*instance par défaut* de n'importe quel module de classe : comme un objet global *sans état* qui représente le *type de classe* lui-même — et qui ne devrait jamais être affiché à l'utilisateur. Quand on doit afficher un formulaire, on crée une *nouvelle* instance de la classe et on affiche *cette* instance.

Le code d'UI peut rapidement devenir difficile à maintenir, parce qu'il est très facile de mélanger les responsabilités : le formulaire se retrouve responsable non seulement d'afficher des données, mais aussi d'*agir* sur elles, et parfois même d'orchestrer l'ensemble du processus. Si on implémente la fonctionnalité directement dans les gestionnaires d'événements Click des boutons, on se retrouve avec beaucoup de « logique métier » qui n'a rien à voir avec l'interface et la présentation. Pour garder les choses propres — et pour faciliter, plus tard, le refactor d'une *Smart UI* vers une architecture plus structurée — la logique métier devrait être extraite dans son propre module dédié.

Même sans adopter une solution orientée objet complète, extraire les *données* du formulaire dans une classe de *modèle*, puis faire en sorte que les contrôles du formulaire manipulent les données de cette classe au lieu de manipuler directement une feuille de calcul, peut faire une grande différence sur la complexité du code d'UI à maintenir et à faire évoluer — que le formulaire soit *modal* ou non.

Un formulaire *modal* est essentiellement une *boîte de dialogue* : il est affiché à l'utilisateur et bloque l'exécution jusqu'à sa fermeture. Il est donc, par nature, transactionnel : proposer des boutons *Annuler* et *Confirmer* a du sens. Si l'utilisateur annule, rien ne se passe. Si l'utilisateur confirme, une procédure prend le modèle et consomme ses données pour effectuer une opération.

Par défaut, les formulaires sont affichés *modally*, mais vous pouvez aussi les afficher en mode non modal. Dans ce cas, le formulaire s'affiche, mais l'exécution continue immédiatement, sans attendre sa fermeture. Gérer une instance non modale qui n'est pas l'*instance par défaut* demande un tout autre paradigme, parce que les interactions utilisateur deviennent asynchrones : on est alors pleinement dans un modèle *piloté par événements*. À l'inverse, avec un formulaire *modal*, la distinction « avant l'affichage » / « après la fermeture » est claire, ce qui rend le comportement plus simple à raisonner.

QUERYCLOSE

Quand vous créez un formulaire modal, il est important de toujours gérer l'événement QueryClose afin d'éviter que l'instance du formulaire ne s'auto-détruisse et ne provoque des problèmes : le bouton [X] de la boîte de contrôle du



formulaire détruit l'objet. C'est déjà un problème si vous n'utilisez pas de *modèle* et que vous interrogez les contrôles du formulaire après sa fermeture... mais, plus généralement, on ne s'attend pas à ce qu'un objet qu'on vient tout juste d'instancier s'auto-détruit spontanément et devienne une référence invalide. Le code qui crée un objet devrait être le code responsable du cycle de vie de cet objet, et un formulaire « auto-destructeur » remet en question cette hypothèse de base.

C'est pourquoi vous voulez faire en sorte que la seule façon de fermer un formulaire soit qu'il se contente de se masquer, ce qui permet de reprendre l'exécution de manière sûre, peu importe comment l'instance a été créée : si c'est l'instance par défaut, vous ne réinitialisez pas son état; si c'est une nouvelle instance, elle n'est pas détruite et le code client (l'appelant) peut consommer son état en toute sécurité, s'il le souhaite.

```
Private Sub UserForm_QueryClose(ByRef Cancel As Integer, ByRef CloseMode As Integer)
    If CloseMode = VbQueryClose.vbFormControlMenu Then
        Cancel = True
        Me.Hide
    End If
End Sub
```

MISE EN GARDE : INTERFACES ACCESS LIÉES AUX DONNÉES

Les projets VBA hébergés dans Microsoft Access peuvent tout à fait utiliser des modules UserForm aussi, mais sans Rubberduck, il faut dénicher la commande correspondante dans l'IDE parce qu'elle est cachée. Cela dit, dans Access, on crée surtout des *Access Forms*, qui (étant des *modules de document* appartenant à l'application hôte) ont beaucoup plus en commun avec un module Worksheet dans Excel qu'avec un UserForm.

Le paradigme est différent dans un formulaire Access, à cause des *liaisons de données* (*data bindings*) : un formulaire lié aux données est intrinsèquement couplé au stockage en base de données sous-jacent, et tout effort visant à découpler l'UI de la base travaille directement *contre* ce que Access cherche à vous simplifier.

Traiter un formulaire Access de la même façon qu'on traiterai une UI de feuille Excel, ça place dans un état d'esprit différent. Imaginez l'UI Battleship sur feuille Excel implémentée comme formulaire Access : le jeu mettrait à jour des enregistrements d'état dans la base sous-jacente, et au lieu d'avoir du code pour tirer l'état du jeu dans l'UI, il suffirait d'avoir du code pour *réinterroger* l'état du jeu, et les liaisons de données se chargeraient de mettre à jour l'UI. Le jeu pourrait même devenir multijoueur : deux clients se connecteraient à la base et partageraient le même état.

C'est fondamentalement différent de la manière dont on injecterait des données dans les contrôles sans liaisons. Lier directement l'UI à une source de données est parfaitement acceptable lorsque cette source s'exécute dans le même processus que celui qui héberge votre code VBA : l'approche *Rapid Application Development* (RAD) d'Access est tout à fait valide dans ce contexte, et ses objets globaux et son état global offrent une API accessible aux débutants pour accomplir énormément de choses, même avec une compréhension minimale du langage (et probablement un peu de SQL Access).

Si l'on parle de formulaires Access *non liés* (*unbound*), alors il vaut probablement la peine d'explorer des architectures *Model-View-Presenter* et *Model-View-Controller* de toute façon : dans ce type de scénarios POO exploratoires, les recommandations ci-dessus peuvent toutes s'appliquer.

EN RÉSUMÉ

- Évitez de travailler directement avec l'*instance par défaut* du formulaire. Créez plutôt une nouvelle instance avec `New`.
- Le module du formulaire (code-behind) devrait se limiter strictement aux préoccupations de *présentation*.
 - Oui, implémentez de la logique d'UI dans le code-behind : par exemple activer un contrôle lorsque la commande dit qu'elle peut être exécutée, afficher un libellé lorsque le modèle est invalide, etc.
- Envisagez de créer une classe de *modèle* pour encapsuler l'état/les données du formulaire.
 - Exposez une propriété en lecture/écriture pour chaque champ modifiable du formulaire.
 - Exposez une propriété en lecture seule pour les données dont les contrôles ont besoin (par ex. les éléments d'une `ListBox`).
 - Les gestionnaires `Change` des contrôles manipulent les propriétés du *modèle*.
 - Exposez d'autres méthodes et propriétés au besoin pour la validation des données/des entrées.
 - Envisagez une propriété `IsValid` qui retourne `True` lorsque toutes les valeurs requises sont présentes et valides, et `False` sinon; utilisez-la pour activer/désactiver le bouton `Accept` du formulaire.
- Évitez d'implémenter une logique à effets de bord dans le gestionnaire `Click` d'un `CommandButton`. Un `CommandButton` devrait *invoquer une commande*, non?
 - En code procédural, la commande peut être une procédure `Public Sub` dans un module standard nommé d'après le formulaire, par exemple un module `SomeDialogCommands` pour un formulaire `SomeDialog`.
 - En POO, la commande peut être une instance (injectée par propriété) d'une classe `DoSomethingCommand` : le gestionnaire `Click` invoque sa méthode `Execute` et peut passer le *modèle* en paramètre.
- Envisagez d'implémenter un objet *présenter* responsable de posséder et d'afficher l'instance du formulaire, surtout pour les formulaires non modaux; le patron d'UI *Model-View-Presenter* est bien documenté, et comme tout en POO, ses concepts ne sont pas spécifiques à un langage ou à une plateforme.
- Masquez le formulaire : ne le laissez pas s'auto-détruire. Évitez de le fermer et de le décharger (*unload*).



CONCEPTION UI

Je ne prétends pas être un gourou du design d'interface, mais au fil des années, je me suis surpris à réutiliser systématiquement les mêmes éléments dans mes formulaires modaux. Et ça a très bien fonctionné pour moi — alors voici comment transformer ça en lignes directrices générales.

UN LOOK PLUS MODERNE AVEC MSFORMS

Quand je conçois un formulaire, je le découpe généralement en trois parties : un panneau en haut, un autre en bas, puis la zone client. Le panneau du haut présente une icône, un titre, et un libellé avec des instructions concises pour l'utilisateur. Le panneau du bas contient au minimum un bouton *OK*, mais le plus souvent j'y mets des boutons *Accept* et *Cancel*. Pour un formulaire non modal, il y aura un bouton *Close*; et si l'action du bouton *OK* peut être exécutée sans fermer le formulaire, j'ajoute un bouton *Apply*. La zone client contient les champs et contrôles du formulaire avec leurs libellés, idéalement dans une mise en page équilibrée qui ne donne pas l'impression d'avoir été bricolée : on y arrive en gardant des marges cohérentes, en alignant les libellés avec les champs, et en évitant les couleurs non système.

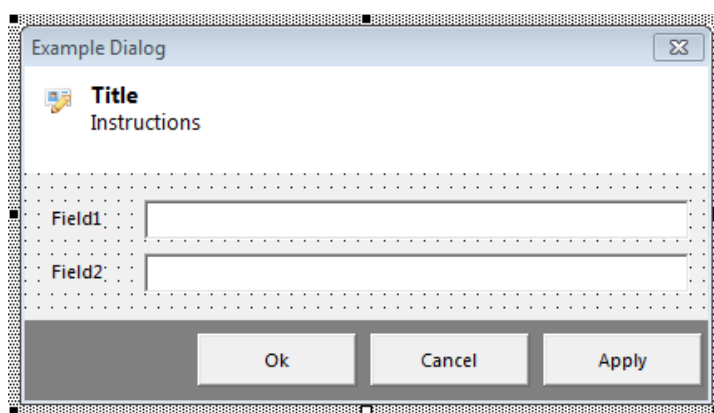


Figure 6 — Exemple de conception visuelle d'un formulaire de dialogue.

- TopPanel est un contrôle Label avec un fond blanc, ancré en haut, et suffisamment haut pour afficher confortablement de courtes instructions.
- BottomPanel est aussi un contrôle Label, avec un fond gris foncé, ancré en bas, et ne dépassant pas 32 pixels de hauteur.
- DialogTitle est un autre contrôle Label avec une police en **gras**, superposé au contrôle TopPanel.
- DialogInstructions est un autre contrôle Label superposé au contrôle TopPanel.
- DialogIcon est un contrôle Image qui affiche une icône .bmp de 16×16 ou 24×24, alignée à gauche, à la même coordonnée Top que le contrôle DialogTitle.
- OkButton, CancelButton, CloseButton, ApplyButton seraient des contrôles CommandButton superposés au contrôle BottomPanel, alignés à droite.

ZONE CLIENT

La mise en page de la zone client n'est pas exactement un « free-for-all », et je doute qu'il soit possible d'établir un ensemble de règles applicables universellement... mais on peut essayer, non?

- Identifiez chaque champ avec un libellé; alignez tous les champs pour donner l'impression d'une grille implicite.
- Cherchez un équilibre visuel; assurez une marge relativement constante sur tous les côtés de la zone client, aérez les éléments, mais pas à l'excès. Utilisez des contrôles Frame pour regrouper des options de ComboBox.
- Évitez de créer un formulaire complexe, avec trop de responsabilités et, inévitablement, trop de contrôles. Au-delà d'un certain seuil de complexité, envisagez plusieurs formulaires distincts plutôt que des onglets.
- Utilisez **Segoe UI** pour une police plus moderne que **MS Sans Serif**, qui était la norme en 1998.
- Ne mettez pas tous les libellés en **gras**.
- Prévoyez une chaîne ToolTip pour le libellé de chaque champ avec lequel l'utilisateur doit interagir. Si un champ est requis ou impose un format/patron particulier, indiquez-le.
- Envisagez d'afficher/masquer une icône 16×16 à côté des champs de saisie (ou même à l'intérieur, alignée à droite) pour indiquer clairement les erreurs de validation. Mettez une info-bulle sur le contrôle Image avec le message d'erreur (p. ex. « ce champ est requis », ou « la valeur ne peut pas dépasser 100 »).
- Nommez. Tout. Ce. Qui. Peut. L'être.
- Réglez le TabIndex de chaque contrôle de façon logique et intuitive, pour que le formulaire puisse être navigué facilement au clavier avec la touche TAB. Appuyer sur TAB dans une zone de texte et voir le focus sauter vers un contrôle sans lien peut être déroutant et frustrant pour l'utilisateur — et c'est un détail auquel on ne pense pas toujours!
- N'utilisez des couleurs de fond dans les contrôles de saisie que pour signaler *très* clairement quelque chose à l'utilisateur — par exemple une erreur de validation qui doit être corrigée pour continuer. Du texte rouge foncé sur un fond rose pâle indique très clairement une valeur invalide.
- Gardez un schéma de couleurs (palette) et un style cohérent dans l'ensemble des composants d'interface de votre application.

Figure 7 — Le formulaire de commande du classeur de suivi des ventes et de l'inventaire de la boutique Ko-fi (aujourd'hui fermée) devait évidemment être en VBA. Celui-ci est réalisé avec MVVM-Lite (voir MVVM dans les suppléments).



Suppléments

SUPPLÉMENTS

MOTIFS ARCHITECTURAUX

Lorsqu'un programme VBA a besoin d'une interface utilisateur, laisser le formulaire gérer « ce qui se passe » revient à tomber dans le [anti-]pattern de la *Smart UI* : ça fonctionne bien pour un prototype rapide, mais ça ne s'adapte pas bien à la croissance du code et peut vite tourner au cauchemar.

C'est là que la POO commence vraiment à briller en VBA : elle vous permet de sortir du moule et de bâtir votre application selon les mêmes paradigmes que ceux utilisés dans des applications « réelles », généralement écrites dans des langages plus centrés POO comme VB.NET, Java ou C#.

Les *patrons de conception* sont parfois un piège pour débutants en programmation orientée objet, parce qu'on peut être tenté de les implémenter « pour le patron ». Pourtant, ce sont surtout des *outils de communication* : ils signalent aux autres développeurs que vous savez ce que vous faites — et pourquoi. Ce sont des manières reconnaissables et distinctives d'organiser les responsabilités des classes et des abstractions. Dans le cas des *motifs architecturaux*, ces patrons permettent de découpler entièrement l'interface utilisateur du reste du code, rendant possible l'automatisation de tests approfondis de la logique applicative *sans* devoir manipuler une UI pour couvrir tous les états possibles.

Utiliser ces patrons exige évidemment une bonne maîtrise des concepts autour des classes et des interfaces; ce sont donc des sujets avancés qui peuvent sembler intimidants, voire carrément excessifs. Vous n'avez pas *besoin* de la POO ni des patrons de conception pour obtenir une solution fonctionnelle!

Mais si vous avez envie d'explorer plus loin, de repousser les limites de ce qu'on peut faire avec VBA, ou d'écrire du code qui pourra être plus facilement porté vers un autre langage un jour, alors c'est une excellente voie.

Essayez de penser votre programme en termes d'interfaces et d'API, et en termes de *couches* dont chacune a ses préoccupations distinctes, ses entrées, ses traitements et ses sorties. Pour moi, la partie la plus intéressante, c'est l'interface / l'API : c'est là que le code exprime réellement ce que vous voulez accomplir. Le nom de chaque méthode et propriété, les hypothèses que vous intégrez (ou contre lesquelles vous vous protégez), les erreurs que vous levez quand ça tourne mal — ça, c'est votre API. À l'intérieur des méthodes, le code devrait être plutôt trivial, facile à comprendre : 5–10 lignes, 20 au maximum. Si c'est plus long, c'est qu'il manque probablement un niveau d'abstraction ou une dépendance; le *Single Responsibility Principle* est peut-être mis à mal. En extrayant des choses dans des portées plus petites, et en prenant le temps de bien les nommer, les abstractions finissent souvent par émerger par nécessité — par exemple lorsqu'on cherche à réduire la répétition (*don't repeat yourself / DRY*).

C'est comme ça que des patrons tels que *Repository* et *Unit of Work* (et bien d'autres) sont apparus : des problèmes qui reviennent sans cesse finissent par faire émerger, à chaque fois, des abstractions similaires.

Dans le cas des *motifs architecturaux*, il est important de reconnaître qu'ils sont pensés pour le *développement logiciel* et que, dans le monde réel, la plupart des projets VBA ne le sont pas (et n'ont pas besoin de l'être).

Mais cela ne veut pas dire qu'ils *ne peuvent pas* l'être.

MODEL-VIEW-PRESENTER (MVP)

En VBA comme en VB6, le framework UI avec lequel on travaille est *Microsoft Forms* (MSForms), l'ancêtre direct de *Windows Forms* dans .NET. Même si la version .NET est nettement plus moderne et plus capable, les deux reposent sur les mêmes mécanismes fondamentaux et fonctionnent de manière très similaire. Le patron d'UI le plus souvent recommandé pour *Windows Forms* est *Model-View-Presenter* (MVP), qui sépare les responsabilités en trois abstractions distinctes :

- La *View* est une abstraction du formulaire lui-même.
- Le *Model* représente les données que le formulaire doit manipuler.
- Le *Presenter* orchestre la logique, communique avec le modèle et synchronise l'état avec la vue.

Ce patron fonctionne très bien avec *Windows Forms* et il n'y a aucune raison qu'il ne fonctionne pas tout aussi bien avec l'ancien framework *Microsoft Forms* : tout ce que vous pouvez lire en ligne sur ce patron peut être transposé en VBA. Comme pour les autres patrons du même genre, la clé est de déplacer systématiquement les fonctionnalités hors de la couche UI : le *presenter* est responsable de communiquer avec la couche *model* et de faire circuler les données vers et depuis la *view*. C'est difficile d'y voir clair sans un exemple concret, alors posons un scénario avec de vraies exigences : disons qu'on veut demander à l'utilisateur sa *taille* et son *poids*, calculer un IMC, et écrire le résultat quelque part. Pour compliquer un peu les choses, supposons que l'utilisateur peut saisir les mesures en unités impériales ou métriques (sans que ce soit *trop* complexe : si la taille est métrique, le poids l'est aussi, et inversement).

MODÈLE

Le *model*, ici, correspond aux entrées utilisateur : on sait donc déjà ce dont on a besoin :

- HeightValue As Double
- WeightValue As Double
- UnitType As UnitType

Notre modèle est donc un module de classe (appelons-le *BodyMassCalcModel*) qui pourrait ressembler à ceci :

```
'@Description "Représente les données nécessaires au calcul d'une valeur d'IMC"
Option Explicit

Public Enum MeasurementUnitType
    Metric
    Imperial
End Enum

Public HeightValue As Double
Public WeightValue As Double
Public UnitType As MeasurementUnitType
```

À partir de là, on voudra pouvoir convertir dans un sens ou dans l'autre (un utilisateur peut hésiter!), donc on ajoute des méthodes pour ça :

```
'@Description "Représente les données nécessaires au calcul d'une valeur d'IMC"
Option Explicit

Private Const CentimetersPerInch = 2.54
Private Const PoundsPerKilogram = 2.20462262
```

```

Public Enum MeasurementUnitType
    Metric
    Imperial
End Enum

Public HeightValue As Double
Public WeightValue As Double
Public UnitType As MeasurementUnitType

Public Sub ToMetric()
    If UnitType = MeasurementUnitType.Metric Then Exit Sub
    Debug.Assert UnitType = MeasurementUnitType.Imperial ' vérifier les hypothèses
    HeightValue = HeightValue * CentimetersPerInch
    WeightValue = WeightValue / PoundsPerKilogram
End Sub

Public Sub ToImperial()
    If UnitType = MeasurementUnitType.Imperial Then Exit Sub
    Debug.Assert UnitType = MeasurementUnitType.Metric ' vérifier les hypothèses
    HeightValue = HeightValue / CentimetersPerInch
    WeightValue = WeightValue * PoundsPerKilogram
End Sub

```

VUE

La *view* sera un module de classe `UserForm`, qui vient avec une *instance pré-déclarée* : vous pourriez donc en faire un objet avec état et faire `UserForm1.Show` si vous le souhaitez. Mais avec *Model-View-Presenter*, afficher le formulaire seul ne fera rien, parce que le rôle d'une *view* n'est pas de *faire* quoi que ce soit.

La *view* est un dispositif d'E/S qui envoie et reçoit de l'information vers et depuis l'utilisateur par différents moyens. Elle conserve une référence vers le *model* et en manipule l'état, tout en maintenant un *état visuel* cohérent : le code-behind d'un module `UserForm` est donc surtout responsable de synchroniser l'état entre le modèle et ce qui est présenté à l'écran.

Le formulaire pourrait contenir deux zones de texte avec libellés, et des boutons radio pour sélectionner le système d'unités. Comme on l'affichera de façon modale, il y aura des boutons OK et Cancel... et cela signifie qu'on peut ajouter un champ public `IsCancelled As Boolean` à notre classe de modèle pour suivre l'état d'annulation de la vue.

Le code-behind devrait se lire comme du code ennuyeux, qui ne peut pas mal tourner : on gère les événements des contrôles et on synchronise les propriétés du modèle, point final.

```

Option Explicit
Public Model As BodyMassCalcModel

Private Sub WeightValueBox_Change()
    On Error Resume Next
    Model.WeightValue = CDb1(WeightValueBox.Value)
    On Error GoTo -1
End Sub

Private Sub HeightValueBox_Change()
    On Error Resume Next

```



```

        Model.HeightValue = CDb1(HeightValueBox.Value)
        On Error GoTo -1
    End Sub

    Private Sub OptionMetric_Change()
        If OptionMetric.Value Then
            Model.UnitType = MeasurementUnitType.Metric
        End If
    End Sub

    Private Sub OptionImperial_Change()
        If OptionImperial.Value Then
            Model.UnitType = MeasurementUnitType.Imperial
        End If
    End Sub

```

La raison pour laquelle on désactive temporairement la gestion d'erreurs, c'est qu'on ne sait pas — et on ne peut pas supposer — que les entrées pourront être converties en Double sans problème. On pourrait implémenter divers mécanismes de validation, mais pour les besoins de l'exemple, reprendre l'exécution après l'affectation si la conversion échoue suffit : en cas d'erreur, on affectera implicitement zéro, puis on continuera comme si de rien n'était. On pourrait considérer le modèle invalide si la taille ou le poids vaut zéro, et désactiver le bouton OK dans ce cas — mais n'allons pas là pour l'instant.

Ce qui est important, c'est de Hide l'instance courante du formulaire (Me) et de synchroniser le modèle en cas d'annulation :

```

    Private Sub OkButton_Click()
        Me.Hide
    End Sub

    Private Sub CancelButton_Click()
        OnFormCancelled
    End Sub

    Private Sub OnFormCancelled()
        Model.IsCancelled = True
        Me.Hide
    End Sub

```

L'implémentation de QueryClose peut ensuite invoquer OnFormCancelled elle aussi, afin de synchroniser l'état d'annulation lorsque l'utilisateur ferme le formulaire en cliquant sur le petit « X » dans le coin :

```

    Private Sub UserForm_QueryClose(ByRef Cancel As Integer, ByRef CloseMode As Integer)
        If CloseMode = VbQueryClose.vbFormControlMenu Then
            Cancel = True
            OnFormCancelled
        End If
    End Sub

```

C'est à peu près le seul code qu'on veut dans la vue; on est prêt à l'afficher.

PRÉSENTATEUR

Si on ajoute un nouveau module et qu'on le nomme `BodyMassCalcPresenter` (pour correspondre aux noms de la vue et du modèle), on peut écrire une macro `Show` comme ceci :

```
Option Explicit

Public Sub Show()
    Dim Model As BodyMassCalcModel
    Set Model = New BodyMassCalcModel

    Dim View As BodyMassCalcView
    Set View = New BodyMassCalcView
    Set View.Model = Model

    View.Show
    If Model.IsCancelled Then Exit Sub

    CalculateBMI Model
End Sub

Private Sub CalculateBMI(ByVal Model As BodyMassCalcModel)
    Model.ToMetric
    Debug.Print Model.WeightValue / (Model.HeightValue ^ 2)
End Sub
```

Cet exemple illustre le patron tout en montrant un problème potentiel avec un modèle mutable : le calcul doit s'assurer qu'il travaille avec des valeurs métriques, donc il exécute `Model.ToMetric`. Mais cela introduit un effet de bord : l'état du modèle a changé. Dans ce cas précis, ce n'est peut-être pas grave (l'objet sort de portée à l'instruction suivante), mais on voudra concevoir nos objets de manière plus *encapsulée*, en n'exposant pas les données directement via des champs publics, et en convertissant plutôt en créant et retournant une nouvelle instance au lieu de modifier l'état interne de l'instance courante.



MODEL-VIEW-VIEWMODEL (MVVM)

Dans .NET, MVVM fonctionne très bien avec *Windows Presentation Foundation* (WPF), un framework UI qui utilise du balisage XAML pour décrire la *View*. Les *bindings* XAML peuvent, par exemple, lier le texte d'une zone de texte à une propriété `String` du *ViewModel* (VM). Les boutons se lient à des *commandes* exposées par le VM, et le framework évalue automatiquement si le bouton doit être activé ou non selon que la fonction `CanExecute` de la commande retourne `True` ou `False`.

Les *bindings* de propriétés peuvent être à sens unique (VM→View), à sens unique vers la source (View→VM), bidirectionnels, ou « one-time ». À l'exception de ce dernier, chaque mode implique un mécanisme pour réagir aux changements de propriétés du VM, aux changements de propriétés des contrôles de la *View* (le formulaire), ou aux deux. Dans .NET, cela se résout en implémentant l'interface `INotifyPropertyChanged`, qui impose la présence d'un événement `PropertyChanged`. Le *binding* doit donc s'abonner à cet événement, et c'est au programmeur d'écrire un VM qui implémente correctement l'interface et propage correctement les changements de valeur. Rien de tout cela

n'est impossible en VBA si l'on suit et invoque nous-mêmes nos gestionnaires, puisque les interfaces abstraites ne peuvent pas exposer un Event en VBA.

Le balisage XAML est compilé en instructions .NET; chaque élément XAML (à peu près) correspond à une classe WPF. En théorie, rien de ce que fait le balisage n'est impossible à reproduire en code pur, mais l'inverse n'est pas forcément vrai. La différence, c'est que WPF effectue son propre rendu et contourne la bibliothèque historique `user32` : ses capacités vont bien au-delà de ce que *Windows Forms* peut faire, mais la courbe d'apprentissage est assez abrupte.

Une version « lite » de MVVM peut être obtenue en VBA avec une petite poignée de classes de support pour définir des *bindings* et des notifications de changement de propriété. En implémentant aussi un *command manager* pour gérer des *command bindings*, on peut invoquer la fonction `CanExecute` de la commande liée et, en conséquence, activer ou désactiver le contrôle de bouton associé. Le *command manager* devrait être invoqué lorsque le *binding manager* (qui suit les bindings de données « normaux » vers des propriétés du VM) détermine que l'état courant du VM justifie une réévaluation de `CanExecute` pour chaque commande enregistrée.

Au lieu du balisage XAML, il faut adopter une approche « tout en code » où l'on initialise la *View* (le formulaire) en créant explicitement tous les bindings... et, ensuite, le seul code-behind nécessaire est celui qui concerne strictement la *présentation*. La logique applicative est encapsulée dans des *commandes* liées aux boutons, et les données du modèle sont liées aux autres contrôles du formulaire. Le *Model* correspond aux données et aux moyens de les obtenir (ou de les stocker). Le VM expose les données du modèle dont la *View* a besoin pour afficher — par exemple les éléments d'une combo box. Idéalement, le VM fournit une propriété pour tout ce dont la *View* pourrait avoir besoin de lier : les valeurs de la combo box, mais aussi la sélection courante et le texte descriptif correspondant; le texte du titre du formulaire, le libellé d'instructions, les légendes des boutons... On commence alors à voir à quel point cette approche peut faciliter des sujets comme la *localisation* : si les libellés UI ne sont que des données, ils peuvent être ce qu'on veut à l'exécution.

Bien sûr, en VBA nous n'avons pas tout un framework pour supporter cela, mais on peut tout de même arriver à MVVM en introduisant un petit nombre de classes et d'interfaces clés.

PROPAGATION DES CHANGEMENTS DE PROPRIÉTÉ

Avec MVP, on a vu comment propager l'état de l'UI vers une instance de modèle : on gère les événements des contrôles, puis on affecte la valeur de la propriété correspondante afin de synchroniser le modèle avec l'UI. Mais si on pouvait aussi propager les changements d'état dans l'autre sens? Modifier des propriétés du modèle mettrait automatiquement l'UI à jour! Il faudrait que le modèle expose un Event à lever chaque fois qu'une propriété change, et le formulaire devrait gérer cet événement en déterminant quel contrôle mettre à jour selon la propriété modifiée.

La première étape consiste à reprendre la classe de modèle et à l'*encapsuler* : transformer ses champs publics en vraies propriétés, avec des accesseurs `Get` et `Let`. Si vous utilisez Rubberduck, c'est quelques clics : clic droit sur l'un des champs publics, puis *refactor/encapsulate field* dans le menu Rubberduck, sélectionnez tous les champs, cochez l'option pour générer un `Private Type`, et c'est terminé.

Les procédures `Property Let` s'exécutent lorsqu'on assigne une nouvelle valeur à une propriété : c'est donc là qu'on voudra déclencher le mécanisme de notification (lever un événement) pour signaler le changement de propriété.

On a donc besoin d'un Event. Le problème, c'est que les événements sont un *détail d'implémentation* et ne sont exposés que sur l'interface par défaut d'une classe donnée. Ça ne convient pas, parce qu'on veut une solution qu'on n'ait pas à réinventer de zéro à chaque fois : il nous faut encapsuler la *propagation* des événements dans une abstraction dédiée, puis enregistrer et invoquer manuellement des *callbacks* au lieu de lever et gérer des événements. En MVVM .NET, le *ViewModel* implémente `INotifyPropertyChanged`, ce qui fait qu'un objet expose un

événement `PropertyChanged`. Faisons donc notre propre interface équivalente, que nos ViewModels implémenteront : elle exposera une méthode permettant d'enregistrer un « gestionnaire », ainsi qu'une méthode `NotifyPropertyChanged` qui itérera sur les gestionnaires enregistrés et informera chacun de la propriété modifiée. Pour commencer, on définit l'interface que doivent implémenter les objets pouvant servir de gestionnaires; appelons-la `IHandlePropertyChanged`, avec une méthode `HandlePropertyChanged` :

```
'@Interface
Option Explicit

Public Sub HandlePropertyChanged(ByVal Source As Object, ByVal Name As String)
End Sub
```

Maintenant que les gestionnaires sont définis, on peut définir `INotifyPropertyChanged` comme suit :

```
'@Interface
Option Explicit

Public Sub AddHandler(ByVal Handler As IHandlePropertyChanged)
End Sub

Public Sub OnPropertyChanged(ByVal Source As Object, ByVal Name As String)
End Sub
```

On sait qu'on voudra que notre ViewModel implémente les notifications, mais il manque encore une pièce : une abstraction qui définit un *property binding*, et qui sera le récepteur de ces notifications. Le binding écoutera aussi les événements des contrôles; mais comme chaque type de contrôle a son propre ensemble d'événements, on aura des classes du genre `TextBoxValueBinding` qui implémentent `IHandlePropertyChanged` afin de propager les changements du ViewModel vers l'UI :

```
Option Explicit
Implements IHandlePropertyChanged
Private WithEvents UI As MSForms.TextBox

Private Type TBinding
    Source As Object
    SourceProperty As String
End Type
Private This As TBinding

Public Sub Initialize(ByVal Target As MSForms.TextBox, ByVal Source As Object, ByVal SourceProperty As String)
    Set UI = Target
    Set This.Source = Source
    This.SourceProperty = SourceProperty
    If TypeOf Source Is INotifyPropertyChanged Then RegisterPropertyChanges Source
End Sub

Private Sub RegisterPropertyChanges(ByVal Source As INotifyPropertyChanged)
    Source.AddHandler Me
End Sub
```

```

Private Sub IHandlePropertyChanged_HandlePropertyChanged(ByVal Source As Object, ByVal
Name As String)
    If Source Is This.Source And Name = This.SourceProperty Then
        UI.Text = VBA.Interaction.CallByName(This.Source, This.SourceProperty, VbGet)
    End If
End Sub

Private Sub UI_Change()
    VBA.Interaction.CallByName This.Source, This.SourceProperty, VbLet, UI.Value
End Sub

```

Ensuite, une classe `CheckBoxValueBinding` serait implémentée de manière très similaire :

```

Option Explicit
Implements IHandlePropertyChanged
Private WithEvents UI As MSForms.CheckBox

Private Type TBinding
    Source As Object
    SourceProperty As String
End Type
Private This As TBinding

Public Sub Initialize(ByVal Target As MSForms.CheckBox, ByVal Source As Object, ByVal
SourceProperty As String)
    Set UI = Target
    Set This.Source = Source
    This.SourceProperty = SourceProperty
    If TypeOf Source Is INotifyPropertyChanged Then RegisterPropertyChanges Source
End Sub

Private Sub RegisterPropertyChanges(ByVal Source As INotifyPropertyChanged)
    Source.AddHandler Me
End Sub

Private Sub IHandlePropertyChanged_HandlePropertyChanged(ByVal Source As Object, ByVal
Name As String)
    If Source Is This.Source And Name = This.SourceProperty Then
        UI.Text = VBA.Interaction.CallByName(This.Source, This.SourceProperty, VbGet)
    End If
End Sub

Private Sub UI_Change()
    VBA.Interaction.CallByName This.Source, This.SourceProperty, VbLet, UI.Value
End Sub

```

Comme nous n'avons pas d'autres types de contrôles dans notre UI, il n'est pas nécessaire d'implémenter des classes de binding pour chaque type de contrôle MSForms possible : les bindings se ressemblent tous et remplissent des responsabilités similaires; en créer un nouveau quand on en a besoin ne devrait pas être trop compliqué. Une classe `OneWayPropertyBinding` serait aussi utile :

```

Option Explicit
Implements IHandlePropertyChanged

```



```

Private Type TBinding
    InvertBoolean As Boolean
    Source As Object
    SourceProperty As String
    Target As Object
    TargetProperty As String
End Type

Private This As TBinding

Public Sub Initialize(ByVal Target As MSForms.Control, ByVal TargetProperty As String,
    ByVal Source As Object, ByVal SourceProperty As String, Optional ByVal InvertBoolean
    As Boolean = False)
    Set This.Source = Source
    This.SourceProperty = SourceProperty
    Set This.Target = Target
    This.TargetProperty = TargetProperty
    This.InvertBoolean = InvertBoolean
    If TypeOf Source Is INotifyPropertyChanged Then RegisterPropertyChanges Source
    IHandlePropertyChanged_OnPropertyChanged Source, SourceProperty
End Sub

Private Sub RegisterPropertyChanges(ByVal Source As INotifyPropertyChanged)
    Source.RegisterHandler Me
End Sub

Private Sub IHandlePropertyChanged_HandlePropertyChanged(ByVal Source As Object, ByVal
    Name As String)
    If Source Is This.Source And Name = This.SourceProperty Then
        Dim Value As Variant
        Value = VBA.Interaction.CallByName(This.Source, This.SourceProperty, VbGet)
        If VarType(Value) = vbBoolean And This.InvertBoolean Then
            VBA.Interaction.CallByName This.Target, This.TargetProperty, VbLet, Not
            CBool(Value)
        Else
            VBA.Interaction.CallByName This.Target, This.TargetProperty, VbLet, Value
        End If
    End If
End Sub

```

Il nous faut encore un peu d'infrastructure avant de pouvoir configurer ces bindings. La *View* va vouloir une API expressive et agréable à utiliser pour le faire; ajoutons donc un module `PropertyBindingViewHelper` :

```

Option Explicit

'@Description "Lie une propriété d'un MSForms.Control à une propriété source"
Public Function BindProperty(ByVal Control As MSForms.Control, ByVal ControlProperty
    As String, ByVal SourceProperty As String, ByVal Source As Object, Optional ByVal
    InvertBoolean As Boolean = False) As OneWayPropertyBinding
    Dim Binding As OneWayPropertyBinding
    Set Binding = New OneWayPropertyBinding
    Binding.Initialize Control, ControlProperty, Source, SourceProperty, InvertBoolean

```



```

    Set BindProperty = Binding
End Function

'@Description "Lie le Text/Value d'un MSForms.TextBox à une propriété source"
Public Function BindTextBox(ByVal Control As MSForms.TextBox, ByVal SourceProperty As
String, ByVal Source As Object) As TextBoxValueBinding
    Dim Binding As TextBoxValueBinding
    Set Binding = New TextBoxValueBinding
    Binding.Initialize Control, Source, SourceProperty
    Set BindTextBox = Binding
End Function

'@Description "Lie la Value d'un MSForms.CheckBox à une propriété source Boolean"
Public Function BindCheckBox(ByVal Control As MSForms.CheckBox, ByVal SourceProperty
As String, ByVal Source As Object) As CheckBoxValueBinding
    Dim Binding As CheckBoxValueBinding
    Set Binding = New CheckBoxValueBinding
    Binding.Initialize Control, Source, SourceProperty
    Set BindCheckBox = Binding
End Function

```

Implémenter `INotifyPropertyChanged` amène un peu de code « boilerplate » qu'on voudra éviter de répéter dans chaque `ViewModel`. On ajoute donc une classe `PropertyChangedHelper` :

```

Option Explicit
Private Handlers As VBA.Collection

Public Sub AddHandler(ByVal Handler As IHandlePropertyChanged)
    Handlers.Add Handler
End Sub

Public Sub Notify(ByVal Source As Object, ByVal Name As String)
    Dim Handler As IHandlePropertyChanged
    For Each Handler In Handlers
        Handler.OnPropertyChanged Source, Name
    Next
End Sub

Private Sub Class_Initialize()
    Set Handlers = New VBA.Collection
End Sub

```

De retour dans le `ViewModel`, on peut maintenant implémenter `INotifyPropertyChanged` en modifiant toutes les procédures `Property Let` pour notifier (uniquement) lorsque la valeur change :

```

'@Description "Représente les données nécessaires au calcul d'une valeur d'IMC."
Option Explicit
> Implements INotifyPropertyChanged
> Private PropertyChanges As New PropertyChangedHelper

Public Enum MeasurementUnitType
    Metric = 0
    Imperial = 1

```



```

End Enum

Private Const CentimetersPerInch = 2.54
Private Const PoundsPerKilogram = 2.20462262

Private Type TModel
    HeightValue As Double
    WeightValue As Double
    UnitType As MeasurementUnitType
End Type
Private This As TModel

Public Property Get HeightValue() As Double
    HeightValue = This.HeightValue
End Property

Public Property Let HeightValue(ByVal RHS As Double)
>     If This.HeightValue <> RHS Then
>         This.HeightValue = RHS
>         OnPropertyChanged "HeightValue"
    End If
End Property

Public Property Get WeightValue() As Double
    WeightValue = This.WeightValue
End Property

Public Property Let WeightValue(ByVal RHS As Double)
>     If This.WeightValue <> RHS Then
>         This.WeightValue = RHS
>         OnPropertyChanged "WeightValue"
    End If
End Property

Public Property Get UnitType() As MeasurementUnitType
    UnitType = This.UnitType
End Property

Public Property Let UnitType(ByVal RHS As MeasurementUnitType)
>     If UnitType <> This.UnitType Then
>         This.UnitType = RHS
>         OnPropertyChanged "UnitType"
    End If
End Property

Public Property Get UnitTypes() As Variant
    Static Result() As Variant
    If IsEmpty(Result) Then
        ReDim Result(0 To 1, 0 To 1)
        Result(MeasurementUnitType.Metric, 0) = MeasurementUnitType.Metric
        Result(MeasurementUnitType.Metric, 1) = "Metric"
        Result(MeasurementUnitType.Imperial, 0) = MeasurementUnitType.Imperial
        Result(MeasurementUnitType.Imperial, 1) = "Imperial"
    End If
    UnitTypes = Result
End Property

```



```

Public Sub ToMetric()
    If UnitType = MeasurementUnitType.Metric Then Exit Sub
    Debug.Assert UnitType = MeasurementUnitType.Imperial ' vérifier les hypothèses
>   HeightValue = HeightValue * CentimetersPerInch
    WeightValue = WeightValue / PoundsPerKilogram
End Sub

> Public Sub ToImperial()
    If UnitType = MeasurementUnitType.Imperial Then Exit Sub
    Debug.Assert UnitType = MeasurementUnitType.Metric ' vérifier les hypothèses
    HeightValue = HeightValue / CentimetersPerInch
    WeightValue = WeightValue * PoundsPerKilogram
> End Sub

Private Sub OnPropertyChanged(ByVal Name As String)
    INotifyPropertyChanged_OnPropertyChanged Me, Name
End Sub

Private Sub INotifyPropertyChanged_AddHandler(ByVal Handler As IHandlePropertyChanged)
    PropertyChanges.AddHandler Handler
End Sub

Private Sub INotifyPropertyChanged_OnPropertyChanged(ByVal Source As Object, ByVal
Name As String)
    PropertyChanges.Notify Source, Name
End Sub

```

À ce stade, tout est déjà en place : il ne reste plus qu'à faire en sorte que la *View* assemble le tout. Et c'est là que MVVM commence vraiment à briller : puisque les bindings gèrent à la fois les événements des contrôles et les changements du VM, il n'est plus nécessaire de gérer quoi que ce soit d'autre que *QueryClose* dans le code-behind du formulaire (et les clics des boutons, mais ignorons-les pour l'instant). Cela nettoie *vraiment* le code-behind! Mais où initialiser et configurer les bindings? Si on le fait dans *UserFormInitialize*, l'instance de modèle ne sera pas encore assignée et on aura des problèmes. Si on le fait dans *UserFormActivate*, on risque des initialisations multiples lorsque l'utilisateur change de fenêtre puis revient au formulaire. Il manque donc une interface *IView* qui expose une méthode pour injecter le modèle et afficher le formulaire :

```

'@Interface IView
Option Explicit

Public Sub Show(ByVal Model As Object)
End Sub

```

On peut ensuite l'implémenter dans le code-behind du formulaire : l'interface reçoit un modèle *Object* pour rester générique, mais on peut le *caster* vers un type plus spécifique en le passant à une autre méthode :

```

Private PropertyBindings As New VBA.Collection

Private Sub IView_Show(ByVal Model As Object)
    ConfigureBindings Model
    Me.Show
End Sub

```



```

Private Sub ConfigureBindings(ByVal Model As BodyMassCalcModel)
    Set This.Model = Model
    With PropertyBindings
        .Add PropertyBindingViewHelper.BindTextBox(Me.HeightValueBox, "HeightValue",
This.Model)
        .Add PropertyBindingViewHelper.BindTextBox(Me.WeightValueBox, "WeightValue",
This.Model)
        .Add PropertyBindingViewHelper.BindComboBox(Me.UnitTypeBox, "UnitType",
This.Model)
        .Add PropertyBindingViewHelper.BindProperty(Me.UnitTypeBox, "List",
"UnitTypes", This.Model)
    End With
End Sub

```

Une fois les bindings créés et ajoutés à une collection au niveau instance (pour garder ces objets en portée), c'est terminé : les contrôles du formulaire s'initialisent avec les valeurs actuellement détenues par le modèle, puis les propriétés du modèle se mettront à jour lorsque l'utilisateur saisira ses valeurs.

Et maintenant, on peut parler de lier ces boutons de commande.

LIAISONS DE COMMANDES

WPF définit une interface `ICommand` qui expose deux méthodes : `CanExecute` devrait retourner `True` si la commande peut être exécutée dans l'état courant, et `False` sinon, afin que le framework sache désactiver le bouton lié — et, bien sûr, une méthode `Execute` qui exécute la commande. On peut définir une interface identique en VBA :

```

'@Interface ICommand
Option Explicit

Public Function CanExecute(ByVal Parameter As Object) As Boolean
End Function

Public Sub Execute(ByVal Parameter As Object)
End Sub

```

Pour l'instant, on voudra deux implémentations : une `CancelCommand` qui peut annuler notre formulaire modal, et une `CalculateBMICommand` où l'on consommera le modèle et produira le résultat du calcul. L'annulation doit toujours être une option : la logique `CanExecute` de `CancelCommand` consiste donc simplement à retourner `True`. En revanche, le calcul devrait être désactivé tant que les entrées ne sont pas valides.

On pourrait passer l'instance de `UserForm` comme paramètre de `CancelCommand.Execute`, mais alors on n'aurait pas accès à la logique d'annulation propre à l'implémentation, seulement à `UserForm.Hide`. Et si on passait une instance de `BodyMassCalcView`, la commande ne découplerait plus rien ! Une solution serait d'exposer la logique d'annulation via `IView`, mais ce serait une interface de très haut niveau d'abstraction et toutes les implémentations n'en auraient pas besoin. Une meilleure solution — alignée avec l'*Interface Segregation Principle* — est donc d'introduire une nouvelle interface `ICancellable` exposant une seule méthode `Cancel` :

```

'@Interface ICancellable
Option Explicit

Public Sub Cancel()
End Sub

```

Pour l'implémenter, il suffit de faire en sorte que `ICancellable_Cancel` invoque la méthode privée `OnCancel`, et c'est tout : la commande d'annulation peut maintenant recevoir un objet `ICancellable` en paramètre et invoquer sa méthode `Cancel`, ce qui mettra correctement à jour l'état d'annulation dans le modèle. Et puisque la commande n'est liée à aucune classe spécifique, elle peut être réutilisée avec n'importe quoi qui implémente un jour `ICancellable`!

```
Implements ICommand

Private Function ICommand_CanExecute(ByVal Parameter As Object) As Boolean
    ICommand_CanExecute = True
End Function

Private Sub ICommand_Execute(ByVal Parameter As Object)
    Dim Cancellable As ICancellable
    If TypeOf Parameter Is ICancellable Then Set Cancellable = Parameter
    If Cancellable Is Nothing Then Exit Sub
    Cancellable.Cancel
End Sub
```

La *View* peut alors implémenter `ICancellable` en invoquant sa propre logique d'annulation :

```
Implements IView
Implements ICancellable

Private Sub OnFormCancelled()
    Model.IsCancelled = True
    Me.Hide
End Sub

Private Sub ICancellable_Cancel()
    OnFormCancelled
End Sub
```

La commande de calcul est plus spécifique à ce projet/exemple, et c'est très bien : toutes les commandes n'ont pas à être des abstractions très générales! Comme cette commande a besoin du modèle, elle recevra l'instance `BodyMassCalcModel` en paramètre.

Maintenant qu'on a des commandes, il faut les câbler et les lier aux boutons de commande dans l'UI. On peut utiliser des bindings de propriétés (à sens unique) pour contrôler leurs libellés et leurs info-bulles, mais il nous faut aussi quelque chose qui écoutera l'événement `Click` et réagira en exécutant la commande appropriée. Voici le `CommandBinding`.

```
Option Explicit
Private WithEvents UI As MSForms.CommandButton

Private Type TCmdBinding
    Command As ICommand
    Parameter As Object
End Type
Private This As TCmdBinding
```

```

Public Sub Initialize(ByVal Command As ICommand, ByVal Parameter As Object)
    Set This.Command = Command
    Set This.Parameter = Parameter
End Sub

Public Sub EvaluateCanExecute()
    This.Button.Enabled = This.Command.CanExecute(This.Parameter)
End Sub

Private Sub UI_Click()
    This.Command.Execute This.Parameter
End Sub

```

Il reste un problème à régler : manipuler le modèle déclenche des notifications `PropertyChanged`, mais aucune d'entre elles ne force un appel aux fonctions `CanExecute` des commandes! Une simple collection de bindings de propriétés ne suffit pas : il nous faut un objet `BindingManager` (faute d'un meilleur nom) qui connaît les bindings de commandes et de propriétés, qui peut aussi écouter les changements de propriétés et, en conséquence, réévaluer si les commandes peuvent être exécutées compte tenu du nouvel état du modèle, en invoquant `EvaluateCanExecute` sur chaque commande.

```

Option Explicit
Implements IHandlePropertyChanged
Private PropertyBindings As VBA.Collection
Private CommandBindings As VBA.Collection

Private Class_Initialize()
    Set PropertyBindings = New VBA.Collection
    Set CommandBindings = New VBA.Collection
End Sub

Public Sub AddPropertyBinding(ByVal Binding As Object)
    PropertyBindings.Add Binding
    If TypeOf Binding Is INotifyPropertyChanged Then
        Dim NotifyingBinding As INotifyPropertyChanged
        Set NotifyingBinding = Binding
        NotifyingBinding.AddHandler Me
    End If
End Sub

Public Sub AddCommandBinding(ByVal Binding As CommandBinding)
    CommandBindings.Add Binding
End Sub

Private Sub IHandlePropertyChanged_HandlePropertyChanged(ByVal Source As Object, ByVal Name As String)
    Dim Binding As CommandBinding
    For Each Binding In CommandBindings
        Binding.EvaluateCanExecute Source
    Next
End Sub

```

La *View* peut alors avoir un champ `Private Bindings As BindingManager`. En enregistrant l'instance `BindingManager` comme gestionnaire des notifications d'un binding, on peut y répondre et réévaluer si les commandes peuvent être exécutées.

Il ne reste plus qu'à implémenter `INotifyPropertyChanged` dans les classes de binding, afin de relayer les changements de propriétés au gestionnaire!

Comme vous pouvez le constater, MVVM exige pas mal d'infrastructure pour fonctionner; et plus on veut supporter de fonctionnalités, plus il faudra de code d'infrastructure : MVVM « veut vraiment » être une bibliothèque qui définit toute cette infrastructure, afin que nous n'ayons plus qu'à l'utiliser.

C'est tout de même un exercice très plaisant, et faire fonctionner MVVM en VBA est extrêmement gratifiant. Certains détails d'implémentation dans le projet du calculateur d'IMC ci-dessus ont été laissés au lecteur, à compléter.

CONCLUSIONS

En raison du code « boilerplate » nécessaire et malgré le fait que MVVM soit très amusant à utiliser, le patron de conception d'UI à privilégier en VBA devrait être MVP / *Model-View-Presenter*. Il peut être implémenté sans aucun boilerplate, avec un modèle minimal qui n'a même pas besoin d'encapsuler des champs publics... même si, on peut le soutenir, il devrait le faire.

À des fins académiques/d'apprentissage, MVVM reste toutefois un excellent prétexte pour apprendre de nombreuses techniques.

Le but de ces patrons est de découpler le modèle et la logique de la vue, afin que le modèle et la logique puissent être testés sans la vue. Qu'il y ait ou non de véritables tests unitaires qui couvrent ce code, ce qui compte, c'est qu'il soit au moins *testable*.



TESTS UNITAIRES

Un test unitaire est une petite procédure simple qui a 3 responsabilités :

1. **Arrange** — préparer les dépendances et définir les attentes.
2. **Act** — exécuter la méthode ou la fonction testée.
3. **Assert** — vérifier que le résultat attendu correspond au résultat obtenu.

C'est pourquoi le gabarit d'une méthode de test Rubberduck ressemble à ceci :

```
'@TestMethod("Uncategorized")
Private Sub TestMethod1() 'TODO Renommer le test
    On Error GoTo TestFail

>     'Préparer :
>     'Exécuter :
>     'Vérifier :
    Assert.Succeed

    TestExit:
    '@Ignore UnhandledOnErrorResumeNext
    On Error Resume Next

    Exit Sub
    TestFail:
    Assert.Fail "Le test a levé une erreur : #" & Err.Number & " - " & Err.Description
    Resume TestExit
End Sub
```

Extrait 1 — Le gabarit standard d'une méthode de test s'assure que toute erreur levée pendant son exécution fait échouer le test avec un message d'erreur relativement convivial. L'appel à `Succeed` est présent uniquement pour s'assurer qu'un test exécute toujours au moins un appel à un membre `Assert`, sinon le test serait jugé non concluant. C'est l'annotation `@TestMethod` (et l'annotation `@TestModule` en haut d'un module standard) qui rend une procédure détectable comme test unitaire Rubberduck.

Lorsqu'un test unitaire s'exécute, Rubberduck suit les appels de méthodes `Assert.Xxxx` (en gras ci-dessus) et leur résultat; si un seul appel `Assert` échoue (le gabarit suggère et encourage un seul *assert* dans le chemin d'exécution normal, mais il peut y en avoir plusieurs), le test échoue. Ces tests automatisés sont très utiles pour documenter les exigences d'une classe *modèle* particulière, ou le comportement d'une fonction utilitaire donnée avec plusieurs paramètres optionnels. Avec une *couverture* suffisante, les tests peuvent prévenir activement l'introduction involontaire de *bogues de régression* lorsque le code est maintenu et modifié : si un ajustement brise un test, tu sais exactement quelle fonctionnalité tu as brisée, et si tous les tests sont au vert, tu sais que le code va encore se comporter comme prévu.

Aie un module de tests par unité/classe que tu testes, et envisage de nommer les méthodes de test selon un patron `MethodUnderTest_GivenAbcThenXyz`, où `MethodUnderTest` est le nom de la méthode testée, `Abc` est une condition particulière, et `Xyz` est le résultat attendu. Pour les tests qui s'attendent à une erreur, envisage un patron `MethodUnderTest_GivenAbc_Throws`. Rubberduck n'avertira pas au sujet des traits de soulignement dans les noms des méthodes de test, et ils sont sûrs parce que les modules de tests Rubberduck sont des modules standards, et les recommandations de nommage pour les tests favorisent généralement une description claire plutôt que la concision.

QUOI TESTER?

Tu veux tester l'*interface publique* de chaque objet, et traiter les membres Private d'un objet comme des *détails d'implémentation*. Tu ne veux PAS tester les *détails d'implémentation*. Par exemple, si l'*interface par défaut* d'une classe n'expose qu'une poignée de membres Property Get et une méthode de fabrique Create qui fait de l'injection de propriétés, alors pour tester cette méthode de fabrique, il devrait y avoir des tests qui valident que chacun des paramètres de Create correspond à une propriété injectée. Si un des paramètres ne peut pas être Nothing, il devrait y avoir une *clause de garde* dans Create pour ce paramètre, et un test unitaire qui s'assure qu'une erreur précise est levée lorsque Create est invoquée avec Nothing pour ce paramètre.

Ci-dessous, un exemple simple de ce type, avec 2 propriétés et une méthode; note comment des tests pour la fonction privée InjectDependencies seraient redondants si la fonction publique Create est déjà couverte :

InjectDependencies est un *détail d'implémentation* de Create :

```
'@PredeclaredId
Option Explicit
Implements IClass1

Private Type TState
    SomeValue As String
    SomeDependency As Object
End Type
Private This As TState

Public Function Create(ByVal SomeValue As String, ByVal SomeDependency As Object) As IClass1
    If SomeValue = vbNullString Then Err.Raise 5
    If SomeDependency Is Nothing Then Err.Raise 5
    Dim Result As Class1
    Set Result = New Class1
    InjectProperties Result, SomeValue, SomeDependency
    Set Create = Result
End Function

Private Sub InjectProperties(ByVal Instance As Class1, ByVal SomeValue As String,
ByVal SomeDependency As Object)
    Instance.SomeValue = SomeValue
    Set Instance.SomeDependency = SomeDependency
End Sub

Public Property Get SomeValue() As String
    SomeValue = This.SomeValue
End Property

Public Property Let SomeValue(ByVal RHS As String)
    This.SomeValue = RHS
End Property

Public Property Get SomeDependency() As Object
    SomeDependency = This.SomeDependency
End Property
```

```
Public Property Set SomeDependency(ByVal RHS As Object)
    Set This.SomeDependency = RHS
End Property

Private Property Get IClass1_SomeValue() As String
    IClass1_SomeValue = This.SomeValue
End Property

Private Property Get IClass1_SomeDependency() As Object
    IClass1_SomeDependency = This.SomeDependency
End Property
```

QU'EST-CE QUI EST *TESTABLE*?

Sans les membres Property Get de Class1 et/ou IClass1, on ne pourrait pas tester que Create injecte par propriété SomeValue et SomeDependency, parce que l'état interne de l'objet est *encapsulé* (comme il se doit). Par conséquent, il y a une hypothèse implicite : un membre Property Get pour une dépendance injectée par propriété retourne la valeur ou la référence encapsulée, et rien de plus; en écrivant des tests qui reposent sur cette hypothèse, on la documente. Note que ce niveau de couverture peut être franchement *exagéré* selon tes besoins; il existe un niveau d'*hypothèses raisonnables* parfaitement acceptable — pas besoin de couvrir entièrement chaque propriété de chaque classe!

Maintenant, SomeDependency pourrait être une instance d'une autre classe, et cette classe pourrait avoir son propre état encapsulé, ses dépendances et sa logique testable. Un module Class1 plus substantiel pourrait avoir une méthode qui appelle SomeDependency.DoSomething, et les tests pour cette méthode devraient pouvoir affirmer que SomeDependency.DoSomething a été appelée une fois.

Si Class1 n'injectait pas SomeDependency par propriété (par exemple si SomeDependency était plutôt instanciée via New), on ne pourrait pas écrire un tel test, parce que le résultat du test pourrait dépendre du fait qu'une méthode soit appelée sur cette dépendance.

Un exemple simple serait Class1 qui instancie un FileSystemObject pour parcourir les fichiers d'un dossier donné. Dans ce cas, FileSystemObject est une dépendance, et si Class1.DoSomething l'instancie directement, alors chaque fois que Class1.DoSomething est appelée, elle va tenter de parcourir les fichiers du dossier, parce que c'est ce que fait un FileSystemObject : ça touche au système de fichiers. Et c'est lent. Les E/S (fichier, réseau, ...et utilisateur) dépendent de tellement de choses qui peuvent mal tourner pour tellement de raisons, que tu voudras généralement éviter qu'elles interfèrent avec les tests.

La façon d'éviter que les entrées/sorties utilisateur, réseau et fichiers interfèrent avec les tests d'une méthode, c'est de *laisser complètement tomber* le "besoin" d'une méthode de *contrôler* ses dépendances. La méthode n'a pas besoin de créer une nouvelle instance d'un FileSystemObject; ce dont elle a vraiment besoin, c'est d'un objet beaucoup plus simple : *n'importe quel objet capable de retourner une liste de fichiers ou de noms de fichiers dans un dossier donné*.

Donc, au lieu de créer une dépendance FileSystemObject directement comme ceci :

```
> Public Sub DoSomething(ByVal Path As String)
    With CreateObjet("Scripting.FileSystemObject")
        ' récupère le dossier Path...
```

```

        ' parcourt tous les fichiers...
        ' ...
    End With
End Sub

```

On aurait plutôt une dépendance sur une abstraction (interface) qu'on pourrait injecter au niveau de l'instance si ça fait du sens, ou au niveau de la méthode en la prenant en paramètre :

```

> Public Sub DoSomething(ByVal Path As String, ByVal FileProvider As IFileProvider)
    Dim Files As Variant
>     Files = FileProvider.GetFiles(Path)
    ' parcourt tous les fichiers...
    ' ...
End Sub

```

Où IFileProvider serait un module interface/classe qui pourrait ressembler à ceci :

```

'@Interface
Option Explicit

Public Function GetFiles(ByVal Path As String) As Variant
End Function

```

Cette interface pourrait très bien être implémentée dans un module de classe nommé FileProvider qui utilise un FileSystemObject pour retourner le tableau promis :

```

Option Explicit
Implements IFileProvider

Private Function IFileProvider_GetFiles(ByVal Path As String) As Variant
    With CreateObject("Scripting.FileSystemObject")
        Dim Folder As Object
        Set Folder = .GetFolder(Path)
        ReDim Result(1 To Folder.Files.Count)

        Dim File As Object, i As Long
        For Each File In Folder.Files
            i = i + 1
            Result(i) = File.Name
        Next
    End With
    IFileProvider_GetFiles = Result
End Function

```

Grâce au *polymorphisme*, elle pourrait aussi être implémentée dans un autre module de classe nommé TestFileProvider, qui utilise un paramètre ParamArray afin que les tests unitaires puissent *prendre le contrôle* de la dépendance IFileProvider et *injecter* (ici par *injection de méthode*) une instance de TestFileProvider. La méthode DoSomething n'a pas besoin de savoir d'où viennent les noms de fichiers, seulement qu'elle peut s'attendre à recevoir un tableau de noms de fichiers existants et valides depuis IFileProvider.GetFiles(String). Si la méthode

DoSomething ne se soucie effectivement pas de la provenance des fichiers, alors elle adhère à pratiquement *tous* les principes de conception OO, et on peut maintenant écrire un test qui échoue si DoSomething *fait quelque chose* de travers — plutôt qu’un test qui échoue parce qu’un lecteur réseau est démonté, ou qui fonctionne localement en télétravail mais seulement avec un VPN.

Le plus difficile est évidemment d’*identifier les dépendances* au départ. Si tu refactorises une macro VBA procédurale, tu dois déterminer quelles sont les entrées et les sorties, quels objets portent l’état qui est modifié, et concevoir une manière de les abstraire et d’injecter ces dépendances depuis le code appelant — que cet appelant soit la macro point d’entrée originale, ou un nouveau test unitaire.

MOCKING VS STUBBING

Dans l’exemple ci-dessus, l’implémentation `TestFileProvider` de la dépendance `IFileProvider` est essentiellement un *stub de test* : tu écris une implémentation distincte uniquement pour pouvoir exécuter le code avec de fausses dépendances qui n’entraînent aucune E/S de fichiers, réseau ou utilisateur. Réutiliser ces stubs dans des macros “test” qui câblent l’interface utilisateur en injectant les stubs de test au lieu des implémentations réelles devrait faire en sorte que l’application fonctionne normalement... sans toucher au système de fichiers ni au réseau.

Avec des *mocks*, tu n’as pas besoin d’écrire une implémentation “test”. À la place, tu configures un objet fourni par un *framework de mocking* pour qu’il se comporte comme le test (ou la méthode testée) en a besoin, et le framework implémente l’interface moquée avec un objet qu’on peut injecter, et dont le comportement correspond vérifiablement à la configuration.

Ça semble magique? Une grande partie l’est, du point de vue VBA/VB6. Beaucoup de tests dans Rubberduck tirent parti d’un framework de mocking très populaire appelé **Moq**. Ce que nous allons publier comme *fonctionnalité expérimentale* n’est pas seulement un wrapper visible COM autour de Moq. Le côté amusant, c’est que les méthodes Moq dont on a besoin sont des méthodes génériques qui prennent des *expressions lambda* en paramètres; notre wrapper doit donc exposer une API que du code VBA peut utiliser, puis la “traduire” en appels de membres vers l’API Moq. Mais comme ce sont des méthodes génériques et que l’interface moquée est un objet COM, on construit essentiellement un type .NET à la volée pour correspondre à l’interface VBA/COM moquée — c’est donc ça que Moq moque réellement : un type d’interface .NET que Rubberduck fabrique à l’exécution à partir de n’importe quel objet COM. Moq utilise la bibliothèque Castle Windsor sous le capot pour générer des instances de *types proxy* — des objets inventés qui implémentent réellement une ou plusieurs interfaces. Castle Windsor est excellent dans ce qu’il fait; on utilise CW pour automatiser l’*injection de dépendances* dans Rubberduck (une technique appelée *Inversion of Control*, où un seul objet *conteneur* est responsable de créer toutes les instances de tous les objets dans l’application au *composition root*; c’est ce qui se passe pendant l’affichage de l’écran de démarrage de Rubberduck).

Il y a toutefois un problème : CW semble mettre en cache les types, avec l’hypothèse raisonnable mais implicite que le type ne changera pas à l’exécution. Dans notre cas, cela signifie que si on moque une interface VBA une fois, puis qu’on modifie cette interface (p. ex. ajouter, enlever, réordonner des membres, ou changer une signature de membre de quelque façon), puis qu’on relance le test, on moque toujours l’ancienne interface tant que le processus hôte est vivant. Ce n’est pas un problème pour moquer une dépendance `Range` ou `Worksheet`, mais le code utilisateur VBA est pénalisé ici.

INVOCATIONS VÉRIFIABLES

En revenant à l’exemple `IFileProvider`, la méthode `GetFiles` pourrait être configurée pour retourner un tableau codé en dur de chaînes bidon, et un test pourrait devenir vert lorsque `IFileProvider.GetFiles` est invoquée avec la même



valeur de paramètre `Path` précise que celle donnée à `Class1.DoSomething`. Si tu faisais du *stubbing* de `IFileProvider`, tu incrémenterais peut-être un compteur chaque fois que `IFileProvider.GetFiles` est appelée, et tu exposerai ce compteur via une propriété que le test pourrait vérifier avec `Assert` pour qu'elle soit égale à une valeur attendue. Avec `Moq`, tu peux faire échouer un test en appelant une méthode `Verify` sur le mock lui-même, qui *vérifie* si la méthode spécifiée a été appelée conformément à la configuration.

Une bonne pratique avec le mocking consiste à ne configurer que le minimum de membres nécessaires pour faire fonctionner le test, à cause du coût en performance : si une interface moquée a 5 méthodes et 3 propriétés, mais que la méthode testée n'a besoin que de 2 méthodes et 1 propriété, alors on ne devrait configurer que celles-là. La vérification rend le mocking très utile pour tester des comportements qui reposent sur des effets de bord et des changements d'état.

Le meilleur, c'est que comme VBA est COM, alors *tout est une interface*; donc si tu n'as pas d'interface `IFileProvider` mais que tu passes quand même un objet `FileProvider` comme dépendance, tu peux moquer `FileProvider` directement et tu n'as pas besoin d'introduire une interface `IFileProvider` “juste pour les tests” si tu n'en as pas déjà une.

API RUBBERDUCK FAKES

Comme le mocking “pur” a rencontré des problèmes, les tests unitaires Rubberduck ont encore un autre outil à leur disposition, qui peut faire une différence et aider à réduire le nombre de wrappers et de stubs de test à implémenter pour pouvoir tester des choses sans que `MsgBox` n'interfère.

En effet, pendant l'exécution d'un test unitaire, Rubberduck peut recevoir l'instruction de se brancher sur le runtime VBA lui-même, d'intercepter certains appels à des fonctions internes spécifiques, et de détourner leurs valeurs de retour pour qu'elles se comportent exactement comme le test en a besoin. Ainsi, tu peux écrire un test pour une méthode qui demande à l'utilisateur s'il veut ou non procéder à une certaine action, instruire la fonction `MsgBox` de retourner `VbMsgBoxResult.vbYes` lorsqu'elle est invoquée, puis vérifier que l'action a été exécutée — ou au contraire lui faire retourner `VbMsgBoxResult.vbNo` et vérifier l'inverse. La *Fakes API* fonctionne de manière semblable à `Moq`, dans le sens où tu peux configurer n'importe laquelle des fonctions détournées pour qu'elle retourne les valeurs nécessaires selon telle ou telle paramétrisation et/ou pour une invocation précise, puis tu peux `Verify` chaque invocation contre le wrapper *fake*.



PATRONS DE CONCEPTION

Cette section présente une petite fraction de patrons de conception courants et leur mise en œuvre respective en VBA. Rubberduck n'est pas nécessaire pour mettre en œuvre quoi que ce soit ici, mais ses outils sont fortement recommandés.

ERREURS ET CLAUSES DE GARDE

Pas exactement un *patron de conception* à proprement parler, mais lorsque vous validez vos paramètres comme toutes premières instructions exécutables d'une procédure et que vous levez une erreur pour *échouer tôt* au lieu de poursuivre et d'échouer peut-être beaucoup plus tard d'une façon qui pourrait être bien plus difficile à déboguer. Ces instructions s'appellent des *clauses de garde*, et leur objectif est précisément d'*échouer tôt* afin de se protéger contre les erreurs de programmation et les entrées invalides en éliminant les hypothèses.

Le plus difficile est d'identifier les hypothèses intégrées à votre code — c'est-à-dire d'anticiper les différentes façons possibles dont vos entrées pourraient être erronées, comme si le code appelant était hostile et cherchait activement à faire exécuter votre code avec des entrées invalides. Si vous recevez une référence d'objet, vous devriez lever une erreur si vous recevez `Nothing`. Si vous récupérez la `Selection` courante, vérifiez que c'est bien un objet `Range` avant de le traiter comme tel. Si vous recevez un `Range`, vérifiez qu'il représente une seule `Area` et qu'il a la forme attendue (lignes, colonnes), ou qu'il représente une seule cellule si c'est ce que votre code attend. Si vous prenez une valeur numérique, vous devriez valider qu'elle est dans l'intervalle de valeurs valides attendu; si des valeurs nulles ou négatives sont inattendues, levez une erreur dès que vous les rencontrez. Des dates qui ne peuvent pas être dans le futur, des chaînes qui ne peuvent pas être vides, des valeurs `Enum` non définies, des appels à un membre d'instance via l'instance par défaut d'une classe alors que cette instance particulière est supposée demeurer sans état, etc.

Avoir un module standard `Guard` avec des procédures `Sub` comme `NotNothing`, `NotEmpty`, `NotInRange`, `NotZero`, `NotNegative`, etc., qui lèvent une erreur d'exécution lorsque la validation échoue, évite de répéter ces vérifications partout (souvent sous des formulations variables) et se lit plutôt bien.

```
Public Sub DoSomething(ByVal Path As String, ByVal FileProvider As IFileProvider)
>     Guard.NotEmpty Path
>     Guard.NotNothing FileProvider
>     '...
End Sub
```

Ce module pourrait exposer un `Public Enum` qui déclare une constante nommée pour chaque erreur personnalisée levée par ce module. Ainsi, au lieu de comparer `Err.Number` à un littéral entier codé en dur, les gestionnaires d'erreurs peuvent le comparer à des constantes nommées et se lire comme `GuardClause.ErrNotNothing` plutôt qu'à un "nombre magique". Comme vous aurez probablement besoin de lever d'autres erreurs d'exécution depuis d'autres modules, gardez en tête les valeurs de l'`Enum` pour éviter qu'elles se chevauchent!

```
Option Explicit

Public Enum GuardClause
    ErrNotNothing = vbObjectError + 1234
    ErrNotEmpty
    ErrNotZero
    '...
End Enum
```



```
Public Sub NotNothing(ByVal Value As Object)
    If Value Is Nothing Then Err.Raise GuardClause.ErrNothing
End Sub

Public Sub NotEmpty(ByVal Value As String)
    If Len(Value) = 0 Then Err.Raise GuardClause.ErrNotEmpty
End Sub

Public Sub NotZero(ByVal Value As Double)
    If Value = 0 Then Err.Raise GuardClause.ErrNotZero
End Sub
```

Il pourrait aussi y avoir un module `Errors` qui servirait à lever des erreurs personnalisées au besoin; cela centralise toutes les erreurs personnalisées levées dans ton application et rend beaucoup plus facile d'éviter les chevauchements, tout en tirant parti des mécanismes de capture d'erreurs en VBA pour tes besoins.

QUAND LEVER UNE ERREUR PERSONNALISÉE?

Si vous laissez du code VBA s'exécuter sans valider les entrées, vous pourriez vous retrouver à devoir déboguer une division par zéro quelque part dans les catacombes d'un module, alors que se protéger contre des entrées invalides vous permet d'*échouer tôt* et de façon *déterministe* : vous savez exactement pourquoi ça échoue à cet endroit du code, et d'où vient l'entrée invalide dans votre pile d'appels si vous vous arrêtez là. Les erreurs personnalisées servent à éliminer les hypothèses qu'une procédure peut faire, même si elles semblent évidentes ou inutiles. Vous levez une erreur personnalisée lorsque l'*état* courant du programme est inattendu et qu'il n'y a pas de récupération possible : le code appelant peut alors décider de gérer l'erreur, ou de la laisser remonter la pile d'appels jusqu'à son propre appelant (la fameuse invite de débogage VBA apparaît quand l'erreur atteint le sommet de la pile d'appels).

ÉTAT PRIVÉ

Si vous suivez ces pratiques depuis un certain temps, vous êtes probablement déjà tombé sur `Private Type TInstanceState` et `Private This As TInstanceState`, où l'on définit un seul champ d'instance privé, de type UDT, qui regroupe tous les champs de stockage ("backing fields") des propriétés de la classe.

Option Explicit

```
Private Type TInstanceState
    SomeValue As String
End Type
> Private This As TInstanceState

Public Property Get SomeValue() As String
>     SomeValue = This.SomeValue
End Property

Public Property Let SomeValue(ByVal Value As String)
>     This.SomeValue = Value
End Property
```

Adopter ce style/patron nettoie la fenêtre d'outils *Locals* du débogueur en reléguant tout l'état privé sous la déclaration `This` (ou quel que soit le nom que vous lui avez donné). Cela a aussi l'avantage de rendre les propriétés limpides en faisant correspondre des noms d'identifiants identiques sans conflit (les champs du UDT doivent être qualifiés).

Rubberduck peut mettre en œuvre ce patron pour vous, via le refactoring *Encapsulate Field*. Dans votre nouveau module de classe, définissez un champ public bien nommé et typé correctement (qu'il soit implicitement public ou non, peu importe) pour chaque propriété que vous souhaitez obtenir; analysez ensuite les modifications, puis faites un clic droit sur l'une d'elles et sélectionnez *refactor/encapsulate field* dans le menu Rubberduck.

Cochez la case pour créer le UDT privé, sélectionnez tous les champs et regardez votre classe s'écrire instantanément; comme Rubberduck modifie le code, il traite automatiquement les changements de code.

FABRIQUES ET FOURNISSEURS

VBA repose sur COM : ses internes sont du COM, et vos projets VBA sont des bibliothèques COM intégrées. Et dans COM, les classes ne peuvent pas avoir de constructeurs paramétrés. On utilise plutôt une *fabrique* pour créer et retourner un objet initialisé. Pour accéder à un système externe ou à un objet qui existe déjà, on parlera plutôt d'un *fournisseur*.

Si vous prenez l'action de créer et d'initialiser un objet et que vous en faites une responsabilité à part entière — une préoccupation distincte — et que vous suivez le *principe de responsabilité unique*, vous pouvez avoir une classe dont le seul rôle est précisément celui-là; on appelle généralement une telle classe une *fabrique*. Toutefois, à moins de mettre en œuvre une *fabrique abstraite*, on a rarement besoin de faire cela, parce qu'une *méthode de fabrique* est beaucoup plus pratique et tout aussi efficace.

MÉTHODE DE FABRIQUE

C'est ce que je considère comme l'approche VBA de l'initialisation d'objets avec paramètres : une méthode Create appelée depuis une *instance par défaut* sans état d'une classe, et qui retourne une nouvelle instance de cette même classe.

Sans Rubberduck, cela peut devenir pénible : l'attribut VB_PredeclaredId est masqué; pour le modifier, vous devez exporter la classe, repérer et ouvrir le fichier avec un éditeur de texte (Notepad, Notepad++) afin de changer la valeur d'attribut de False à True. Ensuite, vous enregistrez et fermez le fichier source .cls et l'éditeur, puis vous importez le fichier modifié dans votre projet. Vous obtenez alors une *instance par défaut globale* de cette classe, accessible partout par son nom; l'instance par défaut d'une classe porte toujours le même nom que le type de la classe. Ainsi, l'instance par défaut de Class1 s'appelle Class1, tout comme l'instance par défaut de UserForm1 s'appelle UserForm1.

Avec Rubberduck, il suffit d'annoter la classe avec @PredeclaredId et de laisser le complément faire le reste.

Si l'on considère cette instance globale gratuite comme "toxique" (comme on devrait probablement le faire : l'état global est généralement à éviter), on évite de l'utiliser pour stocker de l'*état d'instance*; autrement dit, on voudra conserver cette instance par défaut *sans état*. Il existe des façons de se protéger contre les mauvais usages, et c'est ce qui se rapproche le plus, dans une classe VBA, d'un comportement Shared en VB.NET (static en C#), mais sans l'aide à la compilation d'un mot-clé au niveau du langage.

La méthode de fabrique Create est conçue pour être appelée à partir de l'instance par défaut, aux endroits où un opérateur New serait autrement utilisé.

Il est souvent pratique de placer une méthode de fabrique comme premier membre du module de classe, ne serait-ce que comme clin d'œil aux constructeurs paramétrés.

Pour un module Class1, cela pourrait ressembler à ceci :

```
Public Function Create(ByVal Args As Object) As Class1
```

Cependant, comme l'*interface par défaut* de Class1 inclut maintenant une méthode Create, la méthode de fabrique existerait sur l'interface retournée!

La solution consiste à retourner une interface explicite *non par défaut*, que l'on définit dans un module de classe distinct et qu'on pourrait appeler IClass1; le "I" n'est pas strictement nécessaire, mais il indique que la classe est



une interface abstraite tout en rendant la convention de nommage triviale : `I+ClassName` signale qu'on a affaire à une interface clairement destinée à *cette* classe, plutôt qu'à une fonctionnalité quelconque que la classe implémente via une interface.

Cette interface peut exposer des propriétés en lecture seule; si l'on considère les méthodes de fabrique comme l'équivalent VBA des *constructeurs paramétrés*, alors une interface explicite qui expose des propriétés en lecture seule, si elle est correctement implémentée, correspondrait à l'approche VBA des *objets immuables*.

Ainsi, au lieu de retourner `Class1`, on retourne un nouvel objet `Class1`, mais le client ne le voit qu'à travers le prisme de l'interface `IClass1` — et c'est le *polymorphisme* en deux mots!

```
Public Function Create(ByVal Args As Object) As IClass1
```

Dans sa forme la plus simple, sans paramètres, ce patron serait mis en œuvre au moyen d'une fonction sans paramètre qui crée une nouvelle instance de `Class1` et en retourne une référence.

FABRIQUE ABSTRAITE

Si l'on extrait une méthode de fabrique dans sa propre classe et qu'on la fait retourner une interface abstraite, on obtient une implémentation d'une fabrique abstraite. Le patron exige ensuite d'extraire une interface et de consommer cette classe de fabrique via cette interface, de façon à ce que la classe qui la consomme soit complètement découplée de toute classe concrète et ne travaille qu'avec des abstractions. Cela ouvre des possibilités intéressantes : vous pouvez injecter une fabrique qui crée une implémentation complètement différente de l'interface de sortie, et le code client n'y verra que du feu. Ce patron est utile chaque fois que vous ne pouvez pas initialiser entièrement le type de classe retourné au *composition root* où vous effectuez manuellement toute l'injection de dépendances et *composez* votre application.

Ainsi, une fabrique abstraite est essentiellement une interface qui expose une fonction `Create`, qui prend les paramètres que vous souhaitez utiliser pour initialiser le *produit* de la fabrique, et qui retourne une interface abstraite afin de découpler le type concret résultant du code appelant.

Par exemple, une implémentation de `ISomethingFactory_Create` créerait et initialiserait une nouvelle instance de `Something`, mais retournerait un `ISomething`, en gardant le type `Something` comme détail d'implémentation interne de la classe de fabrique.

Ce patron est extrêmement utile avec l'*injection de dépendances*, car il découple non seulement le type de la fabrique, mais aussi le type concret du *produit* de la fabrique : l'objet créé implémente nécessairement l'interface attendue, mais la manière dont il le fait est abstraite. Par exemple, si la fabrique abstraite produit un objet `IFileSystemProvider`, l'implémentation pourrait créer un `FileSystemObject`, mais il pourrait aussi s'agir d'une méthode de test qui injecte un *stub* se contentant de tracer les invocations, sans jamais toucher au système de fichiers; cela ne devrait faire aucune différence pour la classe qui consomme l'objet `IFileSystemProvider` créé.





Annexes

ANNEXES

INDEX

A

Abstraction : interfaces, 11, 12, 32, 34; niveaux, 11, 17, 29, 31

B

Reliure : tôt, 22, 24, 25; tard, 22, 24, 25; propriété, 46

C

Formatage du code : continuations de lignes, 19
Commentaires : annotations, 12, 21, 22;
continuations de ligne, 21; Marque, 21

D

Injection de dépendance, 31, 68; injection
constructeur, 32; injection méthode, 34; injection
de biens, 37
Design Patterns, 41; usine abstraite, 33, 34; méthode
d'usine, 32, 34; modèle-vue-présentateur, 42;
modèle-vue-vue-modèle, 46; interface intelligente,
41

E

Erreurs, 2, 20, 41; temps de compilation, 17, 18;
manipulation, 8, 29; analyse, 6; élévation, 18;

temps d'exécution, 22, 24, 30; niveaux de gravité, 9;
utilisateur, 30; validation, 39
Événements : responsables d'événements, 16, 34, 35
ans; queryclose, 36

I

IntelliSense, 5, 19, 21, 24

P

Paramètres, 29; ByRef, 9, 19, 20, 25, 26; ByRef, 9; Par
Val, 19, 20, 26; Noming, 18, 20; optionnel, 58

R

Refactorisation, 5, 6, 8, 35, 62; interface d'extraction,
32; extraction locale, 19; méthode d'extraction, 6,
29, 31; renommage, 16
Responsabilités, 22, 32, 37, 39, 41, 62, 67; les
appelants, 30, 33, 36; mixte, 35; principe de
responsabilité unique, 32, 34, 41; test, 58; UI, 35

T

Tests : testabilité, 34, 60; Essais unitaires, 5, 32, 41, 58

V

Contrôle de version : git, 21



GLOSSAIRE

- 🦆 **Abstraction** : utiliser des termes abstraits pour dissimuler les détails à la vue. Une variable peut *abstraire* une *expression* ou une valeur; un module peut *abstraire* une fonctionnalité particulière.
- 🦆 **Accessibilité** : détermine si un identifiant peut être consulté au-delà de la portée dans laquelle il est défini, contrôlé par des *modificateurs d'accès* tels que Privé, Public et Ami.
- 🦆 **Annotation** : un commentaire spécial qui commence par @, que Rubberduck interprète comme des métadonnées.
- 🦆 **Argument** : une *expression* évaluée à l'exécution, dont le résultat est transmis au paramètre d'une procédure. Sauf indication contraire, un argument est toujours *positionnel*, et le paramètre correspondant a la même position dans la signature.
- 🦆 **Argument (nommé)** : une autre façon de passer des expressions d'arguments qui utilise l'opérateur spécial := pour permettre de spécifier des arguments dans l'ordre ou pour sauter des paramètres optionnels dans une signature.
- 🦆 **Attribut** : fait référence à des instructions cachées dans des modules de code VBA que VBA interprète comme des métadonnées. Les inspections Rubberduck peuvent détecter les attributs associés à une annotation et synchroniser leurs valeurs.
- 🦆 **Liaison** : dans le contexte de la liaison précoce/tardive, ce terme fait référence à la capacité du compilateur à déterminer l'adresse d'une procédure ou d'une méthode. Dans un contexte d'interface utilisateur, le terme fait référence à la capacité de lier la valeur d'une propriété de contrôle à la valeur d'une propriété d'un autre objet.
- 🦆 **Classe** : un type de données objet défini par un module de classes.
- 🦆 **Client** : l'appelant d'une procédure ou d'une méthode, ou le code qui consomme une classe ou une interface donnée.
- 🦆 **Callback** : une procédure invoquée par l'exécution, comme les gestionnaires d'événements.
- 🦆 **COM** : Le *modèle d'objets composant*; une plateforme héritée pour le développement d'applications Windows, sur laquelle VBA et VB6 sont construits.
- 🦆 **Commande** : un modèle de conception OOP qui abstrait l'exécution d'une commande UI.
- 🦆 **Compilation** : l'acte de traduire des jetons de langue en instructions à faible abstraction.
- 🦆 **Compilateur** : un programme qui analyse le code et le transforme en instructions machine exécutables.
- 🦆 **Composition** : une relation « has-a » entre deux objets où l'un est une dépendance au niveau d'instance de l'autre.
- 🦆 **Flux de contrôle** : instructions qui contrôlent le flux d'exécution, incluant les boucles et conditionnels, les instructions Exit, GoTo/GoSub et Return .
- 🦆 **Type de données** : détermine la quantité de mémoire allouée à une valeur. Par exemple, un entier long alloue 32 octets.
- 🦆 **Injection de dépendances** : une technique qui consiste à isoler les dépendances d'une classe ou d'une procédure, et à fournir ces dépendances du côté client, en *les injectant au niveau de l'instance via l'injection de propriétés (la classe expose une propriété écrivable pour recevoir cette dépendance) en VBA, ou via injection de méthode (une méthode reçoit ses dépendances sous forme de paramètres)*.
- 🦆 **Encapsulation** : la capacité d'un objet à garder l'état interne privé et à fournir une couche d'indirection qui abstrait les détails d'implémentation de cet objet.
- 🦆 **Gestionnaire d'erreur** : une sous-routine qui s'exécute dans le champ d'application d'une procédure lorsqu'une erreur à l'exécution survient, avec une instruction active On Error.
- 🦆 **Gestionnaire d'événements** : une sous-procédure qui possède un nom d'identifiant en deux parties composé du nom du fournisseur d'événement et du nom de l'événement, séparés par un seul soulignement. Invoqué par le VBA lorsque l'événement est *soulevé* par le prestataire.
- 🦆 **Expression** : un terme très générique qui englobe toute opération impliquant un opérateur (où les opérandes sont aussi des expressions elles-mêmes!), y compris des opérateurs unaires comme le . (DOT) Opérateur *d'accès pour les membres*.
- 🦆 **Factory** : un objet responsable de la création d'instances d'un type de classe particulier.
- 🦆 **Factory (méthode)** : une méthode qui crée et retourne une nouvelle instance de la classe dans laquelle elle

est définie, invoquée à partir de l' *instance par défaut de la classe*.

- 🕒 **Factory (abstrait)** : une abstraction (interface) représentant une usine qui crée un type concret mais ne l'expose que comme une interface; découple l'objet usine de la classe qu'il crée.
- 🕒 **Champ (instance)** : une variable au niveau du module dans un module de classe.
- 🕒 **Héritage** : l'un des piliers de la POO représente une relation « is-a » entre deux types de classes, où l'un est *dérivé de l'autre*. Par exemple, une classe ThisWorkbook expose ses propres membres mais hérite aussi de ceux de sa *classe Workbook de base* . L'héritage n'est pas supporté dans VBA; utilisez *plutôt la composition*.
- 🕒 **Instance** : un objet à l'exécution alloué en mémoire, créé à l'aide de l'opérateur Nouveau ou de la fonction CreateObject.
- 🕒 **Interface** : définit [ou fait référence à] membres exposés par un objet.
- 🕒 **Interface (abstrait)** : un module de classe dont *l'interface par défaut* expose des membres destinés à être implémentés par d'autres modules de classes.
- 🕒 **Interface (par défaut)** : les membres publics d'un module de classe. Notez que les champs publics sont exposés comme des propriétés de lecture/écriture.
- 🕒 **Conteneur d'inversion de contrôle (IoC)** : un objet puissant capable d'enregistrer, de résoudre et de libérer un graphe de dépendance. Utilisé dans de nombreux langages pour gérer et automatiser les 3 étapes (registre, résolution, release) de la DI.
- 🕒 **Macro** : une procédure publique généralement sans paramètres, invoquée depuis l'application hôte ou attachée à une forme ou un bouton.
- 🕒 **Membre** : toute déclaration au niveau du module, incluant constantes, variables, importations de bibliothèques et procédures.
- 🕒 **Méthode** : Sous-procédure ou procédure de fonction exposée par une interface (implémentée ou non).
- 🕒 **Objet** : une instance d'exécution d'une classe, présentant une *interface par défaut* définie par un module de classe.

- 🕒 **Orienté objet** : un paradigme de programmation où les objets et leurs interfaces forment les éléments de base d'un programme...
- 🕒 **Paramètre** : une variable locale qui est déclarée avec une syntaxe dédiée dans la signature de la procédure.
- 🕒 **Polymorphisme** : la capacité de présenter la même interface pour différents types d'objets sous-jacents.
- 🕒 **Procédural** : un paradigme de programmation où les procédures et modules forment la base des niveaux d'abstraction.
- 🕒 **Refactorisation** : une opération d'édition qui consiste à modifier la façon dont le code fonctionne, sans modifier ce qu'il fait. Renommer une variable (et mettre à jour toutes ses références) est un exemple de refactorisation simple.
- 🕒 **Portée** : un contexte d'exécution dans lequel un nom d'identifiant donné a une signification spécifique particulière. Il y a 3 champs d'application dans VBA : global/projet, module/instance et local.
- 🕒 **Contrôle de source (version)** : une technologie qui sécurise l'historique et l'auteur du code source et des fichiers texte dans une structure de projet ou de dossiers.
- 🕒 **État (global)** : valeurs accessibles de n'importe où dans le projet, exposées par des modules et/ou des bibliothèques référencées. Inclut les variables dans la portée globale et l' *état d'instance* des objets accessibles globalement, comme Excel.Application.
- 🕒 **État (instance)** : les différentes valeurs détenues par un objet à l'exécution, qu'elles soient internes/privées ou exposées publiquement. L'état de l'instance est toujours muable (peut être écrit par n'importe quel membre de la classe) dans VBA, même si lu (get) uniquement à l'externe.
- 🕒 **Test (unité)** : une petite procédure qui établit des attentes et des dépendances, invoque la méthode testée et compare les attentes aux résultats réels; si l'assertion réussit, le test passe.
- 🕒 **Token** : un « mot » grammatical dans un langage de programmation, par exemple Dim ou End Sub, mais les opérateurs et identifiants sont aussi des jetons.



🦆 **Variable** : un identifiant qui fait référence à une adresse spécifique (ou une plage d'adresses) en mémoire. À moins qu'un *type de données* explicite ne soit spécifié, une déclaration de variable attribue une variante



Auto-publié le 1er juin 2023, sous forme de document PDF téléversé dans la boutique Rubberduck Ko-fi pour 19,99 \$ canadiens (50% de rabais pour les abonnés), article # RDPDFDL001.

Le document est rapidement passé par la suite en mode « payez ce que vous voulez » sous lequel il a été disponible jusqu'à la fermeture de la boutique. Cette ré-édition prévoit poursuivre la distribution de cette façon.



Rubberduck Copyright © 2014-2026 Contributeurs Rubberduck, sous licence GPLv3.
Guide de style Rubberduck Copyright © 2023-2025 Mathieu Guindon, tous droits réservés.
Guide de style Rubberduck Copyright © 2026 9562-7303 Québec inc., tous droits réservés.

L'article original est disponible gratuitement sur <https://rubberduckvba.blog/2021/05/29/rubberduck-style-guide>.

ReSharper (R#) est une marque déposée de JetBrains LLC.; Microsoft, Windows, Microsoft Visual Studio, Excel, Word, Office et d'autres propriétés Microsoft sont des produits et/ou des marques déposées de Microsoft Corporation. Toutes les autres marques de commerce appartiennent à leurs propriétaires respectifs.

